

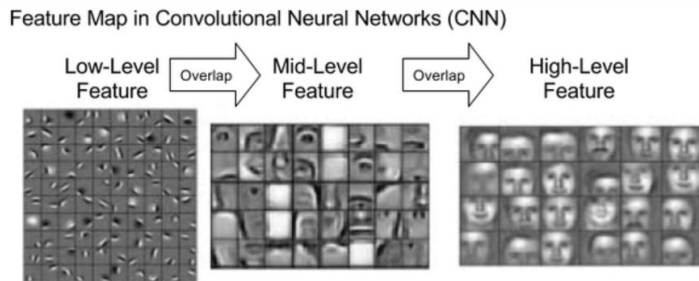
Deep learning part II

Olivier Caelen

Convolution neural network

Convolution neural network

- Like MLP, a convolutional neural network consists of an input layer, hidden layers and an output layer.
- Learn automatically hierarchical representation to detect patterns in the data.
- Convolutional neural networks are state of the art for image classification, and are applied successfully to many other domains (speech analysis, genomics, strategy games, ...)



Credit: MIT Introduction to Deep Learning | 6.S191

Tensor

Tensors are multi-dimensional arrays with a uniform type.

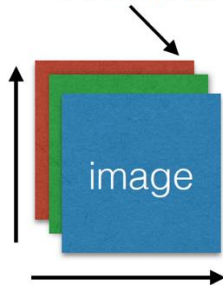
- 0-D : a single number
- 1-D: a vector
- 2-D: a matrix
- 3-D: a 'cube' of numbers
- ...

In a typical convolution neural network for image processing, the input is a tensor with a shape:
(number of inputs) x (input height) x (input width) x (input channels)

```
# shape (60000, 28, 28) -> (60000, 28, 28, 1)
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
```

What is the dimension of this tensor?

C=3 channels



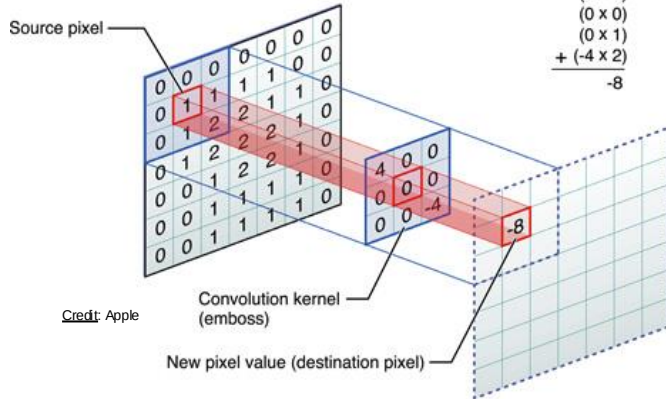
Inputs are represented as **'stacks'** of
2D data, called **channels**

In a convolutional neural network, the hidden layers include layers that perform **convolutions**, and this is followed by other layers such as **pooling** layers, **fully connected** layers.

Convolutional layers

Scalar product of kernel with each position in the input array

Center element of the kernel is placed over the source pixel. The source pixel is then replaced with a weighted sum of itself and nearby pixels.



$$\begin{array}{r}
 (4 \times 0) \\
 (0 \times 0) \\
 (0 \times 0) \\
 (0 \times 0) \\
 (0 \times 1) \\
 (0 \times 1) \\
 (0 \times 0) \\
 (0 \times 1) \\
 + (-4 \times 2) \\
 \hline
 -8
 \end{array}$$

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved Feature

Credit: <https://www.coursera.org/learn/convolutional-neural-networks/home/>

– Act as a pattern detection layer –
This filter detects the crosses

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

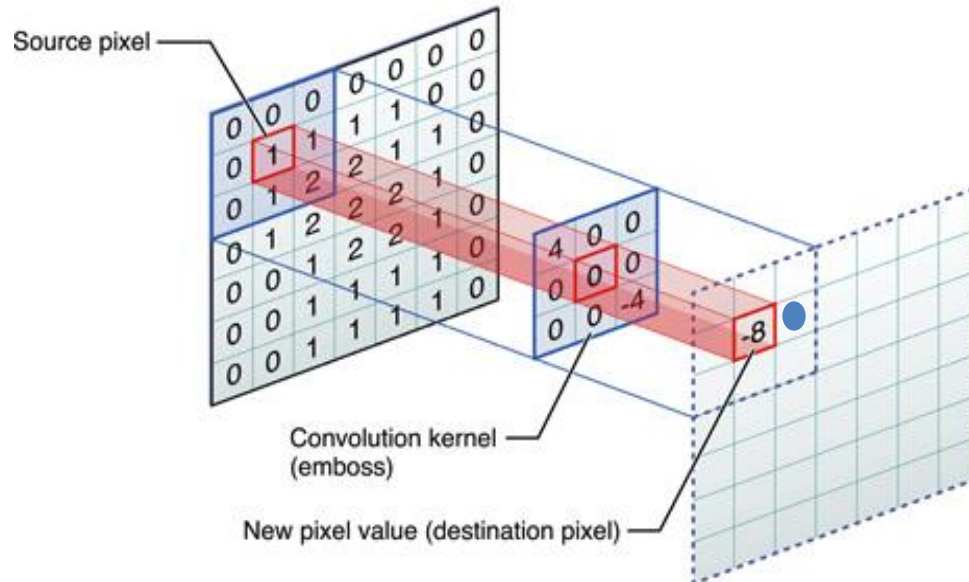
4	3	4
2	4	3
2	3	4

Convolution will give highest scores on the feature map for regions in the input image that match the filter/kernel

A nonlinear activation function (e.g., ReLU) is generally applied after convolution

Question for you:

- What would be the value on the blue dot?
- For which input would the convolution be maximum?



Demo

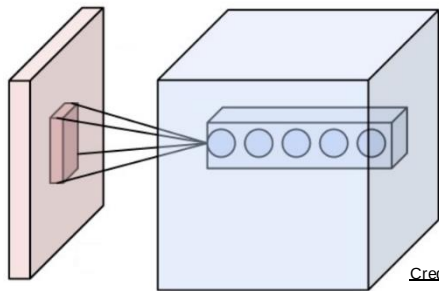
<https://setosa.io/ev/image-kernels/>

Convolutional layers

The convolutional layer is the core building block of a convolution neural network

A convolutional layer has the following main hyper-parameters:

- Convolutional filters/kernels defined by a width and height
- The number of output channels (number of filters/neurons)

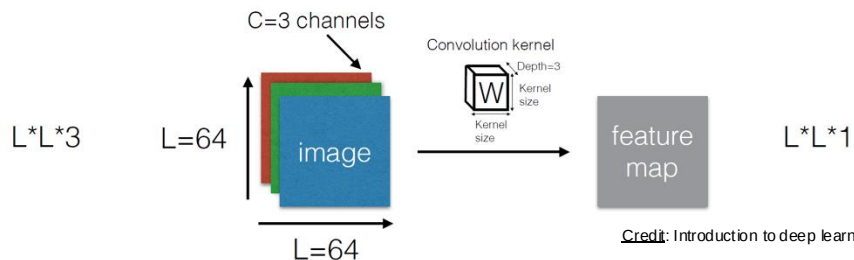


Credit: <https://cs231n.github.io/convolutional-networks/>

- Additional hyperparameters of the convolution operation, such as:
 - Padding
 - Stride
 - Dilation
 - ...

Convolutional layers

- As the FC layer, a conv layer has input, a set of weights, and output.
- However, the weights (called kernel) are not tied to specific neurons.
- A different representation and terminology is therefore often preferred to better reflect the convolution layer purpose
- The convolution of the input array ($L \times L \times C$) by the kernel ($K \times K \times C$) results in an array of size $L \times L \times 1$, called **feature map (FM)**

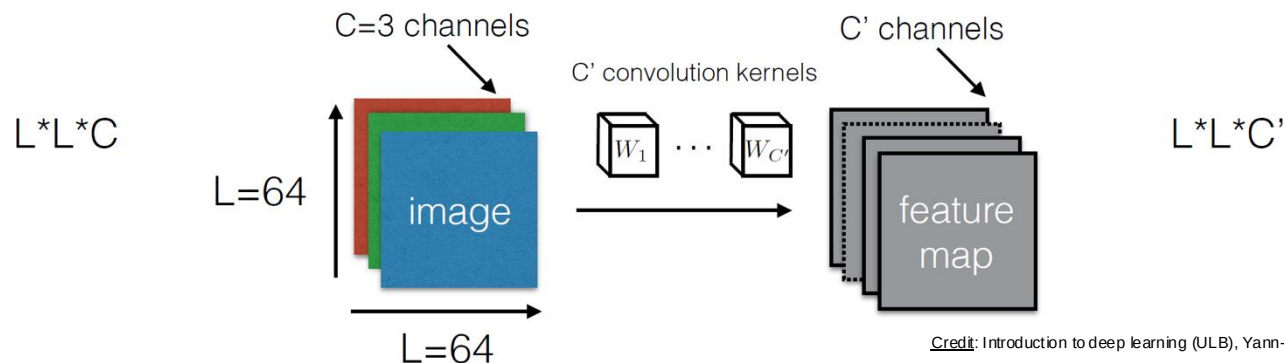


Credit: Introduction to deep learning (ULB), Yann-Aël Le Borgne

- Question: how many parameters in this kernel?

Convolutional layers

- If the input has C channels, the kernel has depth C .
 - Scalar product is extended to the 'depth' of the input.
- The number of kernels determine the number of feature maps (output channels)



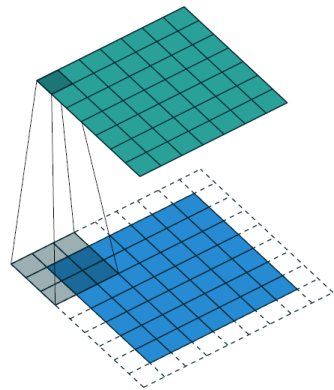
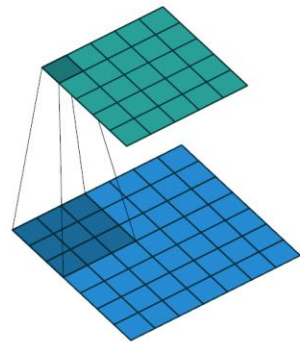
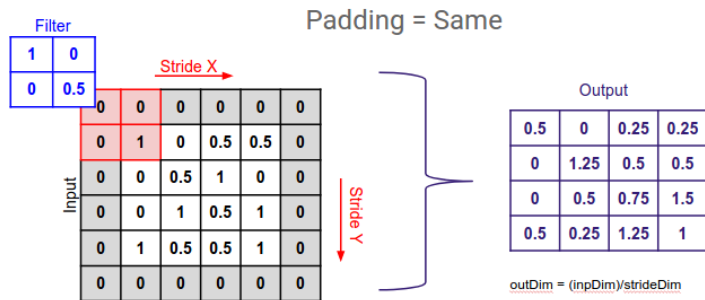
Credit: Introduction to deep learning (ULB), Yann-Aël Le Borgne

Convolutional layers - Padding

If input is 7×7 and kernel 3×3 , what is the size of the output?

Any way to make output size the same as input size?

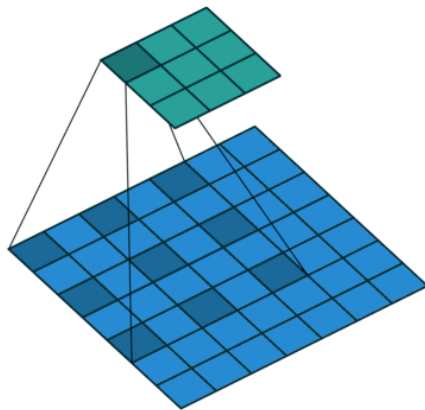
Padding consists in adding extra zeros around input FM so that the dimension of the input FM and output FM is the same.



If kernel is $k \times k$, what should be the padding width?

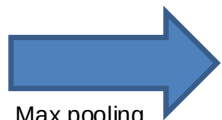
Convolutional layers

- Stride
 - The amount of movement in the image.
 - For example, if the stride is set to 1, the filter will move one pixel at a time.
 - The stride affects the encoded output volume
- Dilation
 - Dilated convolution is the same convolution, only with pixel skipping.

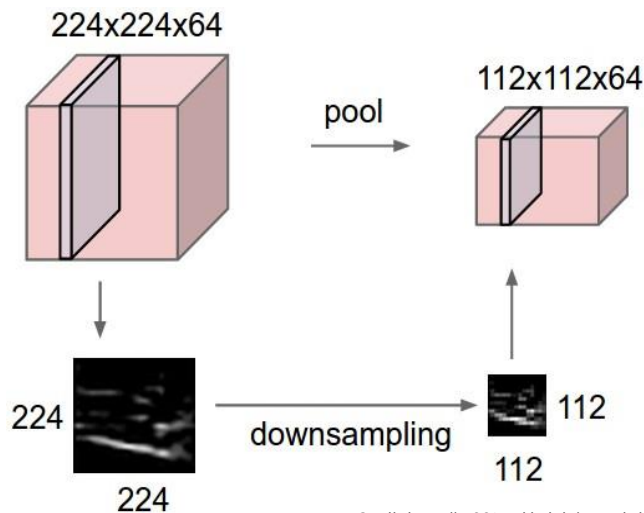


Pooling layers

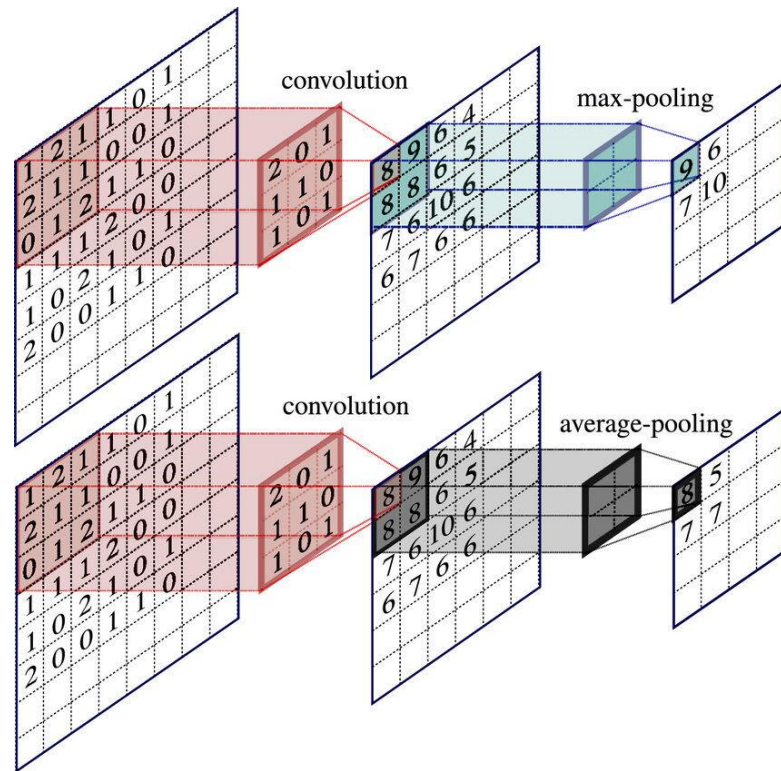
- Reduce the size of the representation to speed up computation as well as make some feature detection.

$$\begin{bmatrix} 0 & 1 & 2 & 5 & 8 & 9 \\ 2 & 2 & 0 & 6 & 6 & 8 \\ 2 & 3 & 3 & 1 & 4 & 3 \\ 3 & 1 & 2 & 2 & 2 & 2 \\ 4 & 6 & 5 & 3 & 2 & 1 \\ 7 & 7 & 5 & 4 & 2 & 0 \end{bmatrix}$$


Max pooling
Size: (2*2)
Stride = 2

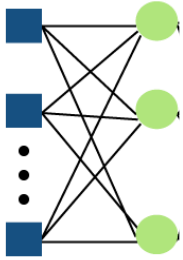
$$\begin{bmatrix} 2 & 6 & 9 \\ 3 & 3 & 4 \\ 7 & 5 & 2 \end{bmatrix}$$


Pooling typically reduce the size of the input block by 4, using the max or mean operators.



Fully connected layers

Same as in MLP



Questions for you!

What happens to the spatial dimension of the output when you increase the stride in a convolutional layer?

- A. The output spatial dimension can increase or decrease.
- B. The output spatial dimension decreases.
- C. The output spatial dimension does not vary unless the number of filters change.
- D. The output spatial dimension increases.

Why are pooling layers often included in convolutional neural networks? Check all that apply (many are possible).

- A. To allow a degree of translational invariance on the input
- B. To increase the number of filters in a layer
- C. To down sample the spatial dimensions, thereby reducing the number of network parameters

AlexNet(2012) Krizhevsky et al.

$(CL \rightarrow MP)^2 \rightarrow CL^3 \rightarrow MP \rightarrow (FC \rightarrow DO)^2 \rightarrow \text{Linear} \rightarrow \text{Softmax}$

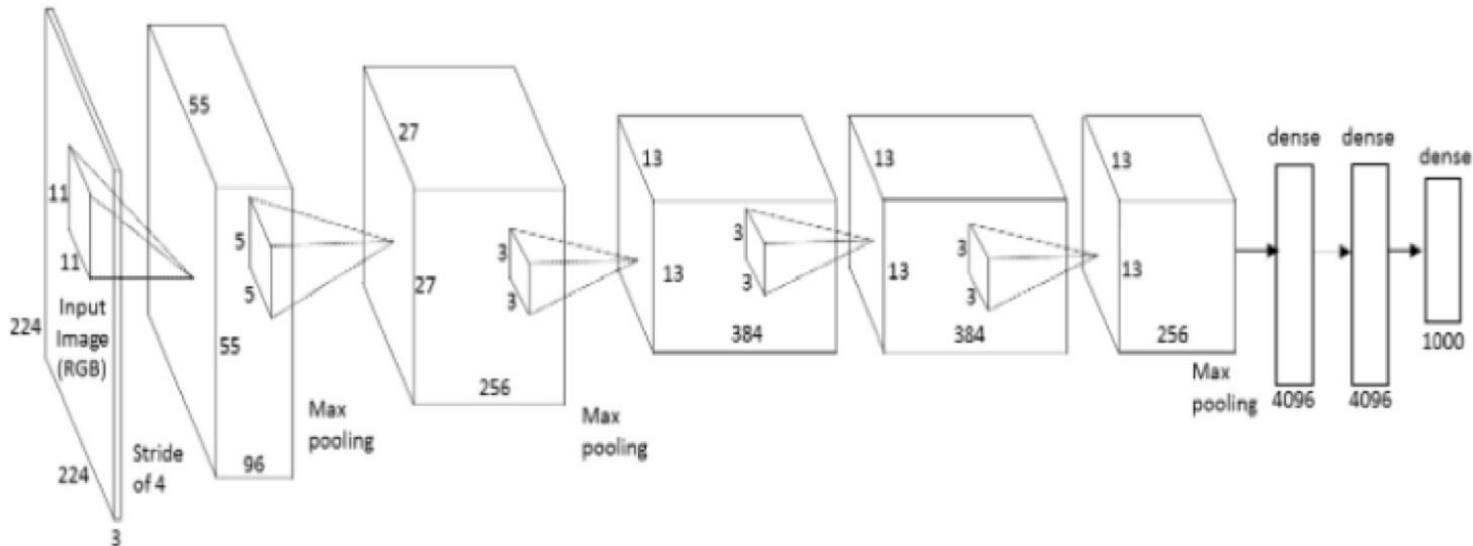
CL: Convolutional layers [with ReLu]

MP: Max pooling

DO: Dropout

FC: Fully connected layer [with ReLU activation]

Linear: Fully connected layer [without activation]



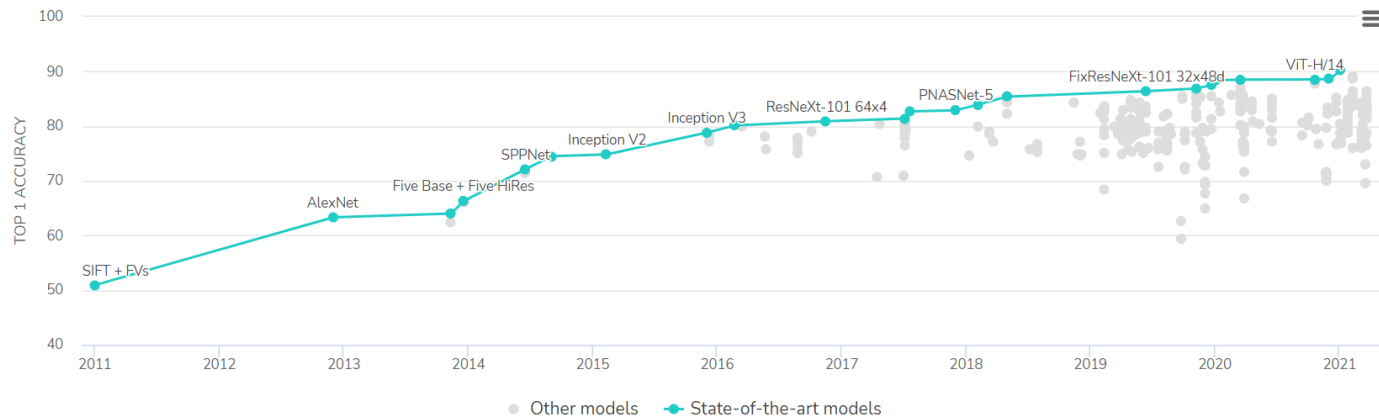
AlexNet

Type	Filter Size / Stride / Padding	Output Size (H × W × D)	Activation Function	Dropout
RGB Image (Input)	-	224 × 224 × 3	-	-
Convolution	11 × 11 / Stride 4 / No Padding	55 × 55 × 96	ReLU	-
Max Pooling	3 × 3 / Stride 2 / No Padding	27 × 27 × 96	-	-
Convolution	5 × 5 / Stride 1 / Padding 2	27 × 27 × 256	ReLU	-
Max Pooling	3 × 3 / Stride 2 / No Padding	13 × 13 × 256	-	-
Convolution	3 × 3 / Stride 1 / Padding 1	13 × 13 × 384	ReLU	-
Convolution	3 × 3 / Stride 1 / Padding 1	13 × 13 × 384	ReLU	-
Convolution	3 × 3 / Stride 1 / Padding 1	13 × 13 × 256	ReLU	-
Max Pooling	3 × 3 / Stride 2 / No Padding	6 × 6 × 256	-	-
Fully Connected	-	4096	ReLU	0.5
Fully Connected	-	4096	ReLU	0.5
Fully Connected	-	1000	Softmax	-

Image Classification on ImageNet

Leaderboard

Dataset



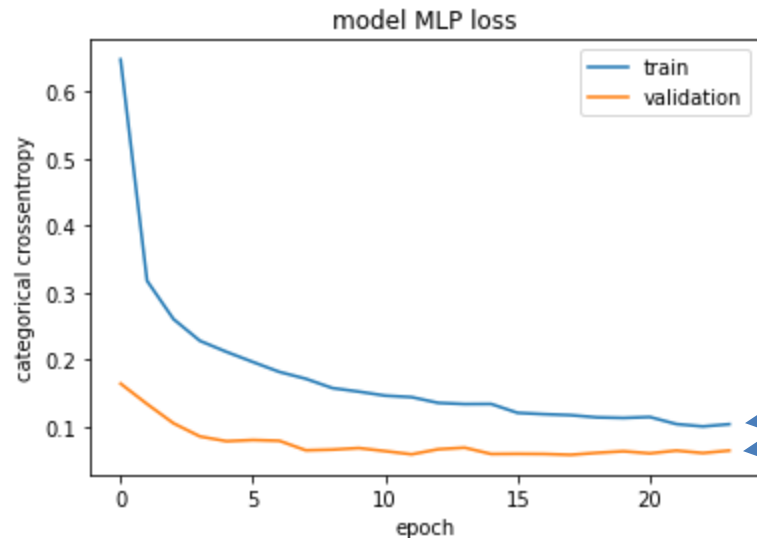
<https://paperswithcode.com/sota/image-classification-on-imagenet>

Example

https://github.com/ocaelen/intro_deeplearning/blob/main/simple_cnn.ipynb

My validation loss is lower than my training loss???

From my previous example, I have this learning curve...



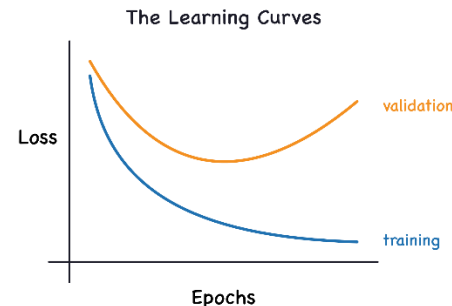
???

???

We observe a higher training loss even though the model learns from this training set.



... but normally from textbooks, it should be some think like this:



Question for you: Do you have any idea why this phenomenon is common with neural networks?
(there are several possible reasons!)

My validation loss is lower than my training loss???

- **L1 or L2 Regularization:**

Regularization terms are only applied during model training on the training set, which inflates the training loss. During validation, your loss function includes only the prediction error, resulting in a loss that is generally lower than the training set.



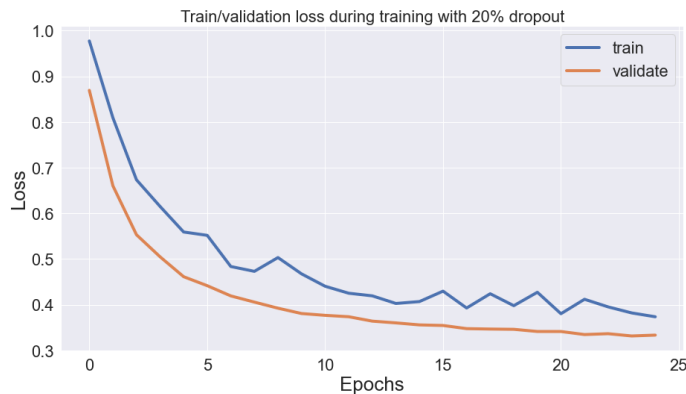
Validation loss is always less than training loss, but the gap between the two narrows over time.

The difference between validation loss and learning loss decreases after each epoch. This is because as the network learns the data, it also reduces the regularization loss (model weight), resulting in a minor difference between validation loss and learning loss.

My validation loss is lower than my training loss???

- **Dropout:**

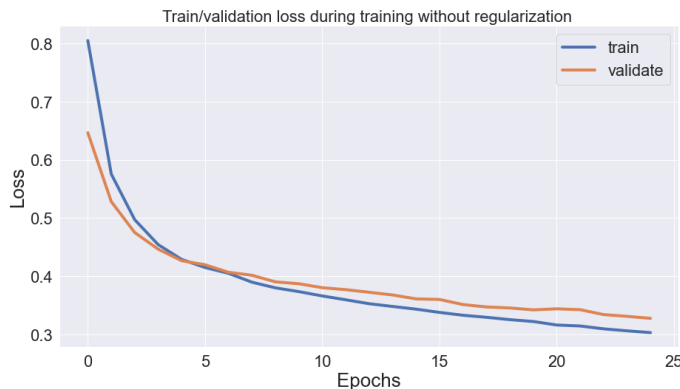
Dropout reduces model variance by randomly freezing neurons in a layer during model training. Dropout is only applicable during the training process and affects the training loss, leading to cases where the validation loss is less than the training loss.



Validation loss is consistently lower than learning loss, the gap between the two remains more or less the same

My validation loss is lower than my training loss???

- **Training loss is calculated during each epoch, but validation loss is calculated at the end of each epoch:**
 - Each epoch is complete when all the training data is run through the network exactly once, and if you run the data in small batches, each epoch can have multiple back-propagation steps.
 - Each backpropagation step could improve the model significantly, especially in the early epochs when the weights are still relatively untrained.
 - As “*training loss is calculated during each epoch, but validation loss is calculated at the end of each epoch*”, you can get a smaller validation loss in the early epochs when each backpropagation updates the model significantly.



My validation loss is lower than my training loss???

- **By luck!**
 - Applicable to all ML models.
 - You don't have that much data.
 - When you do the train/validation split, *by chance* the train set can have examples that are difficult to classify compared to the ones in the validation set.
 - This problem disappears if there are enough examples in both sets.

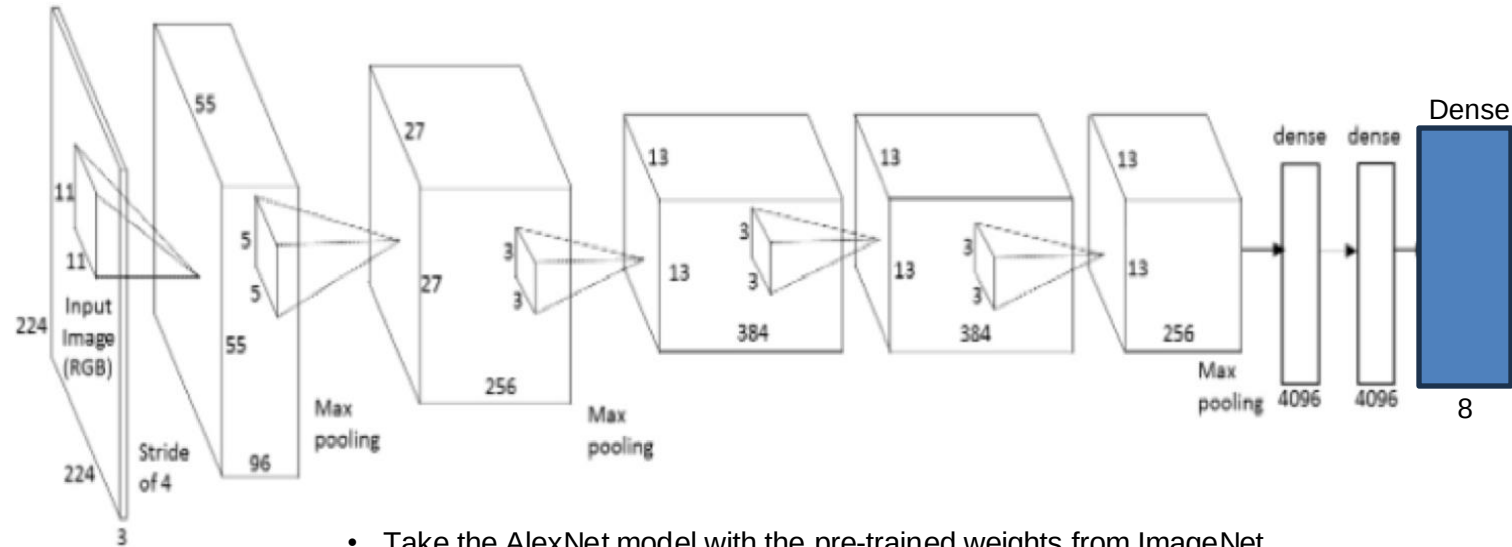
Transfer Learning in Deep Learning

Consider the following scenario: We have a relatively small supervised dataset of annotated images of eight different species of tulips (e.g., 10 images per type, for a total of 80 images). Our objective is to make a classifier capable of identifying the correct type of tulip.



Question for you: How do you build such a model with so few observations?

Transfer Learning in Deep Learning (example with AlexNet)

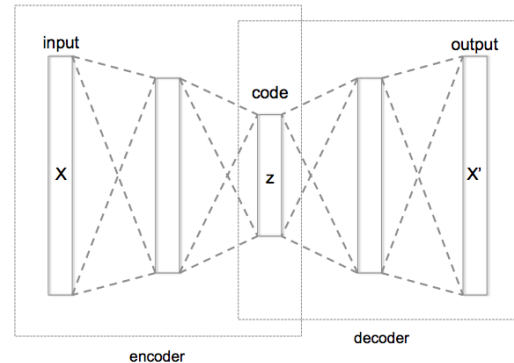


- Take the AlexNet model with the pre-trained weights from ImageNet.
- Remove the last layers (here only the last one).
- Add new layers to the end of the model (here only one new layer).
- Train this new model on the new data (with and without freezing existing pre-trained weights).
- **Warning:**
 - catastrophic forgetting!
 - Input format of new task should be compatible

Deep generative modeling

Autoencoder

- An autoencoder is a type of artificial neural network used to learn efficient data encodings in an unsupervised manner.



- What we care about is the hidden layer at the middle ('code').
- When the NN can reliably reconstruct the output, the hidden layer must contain enough information to represent the output.
- If the hidden layer ('code') is smaller than the input/output, then the information is compressed in a lower dimension.
- Roughly speaking, we set the input equal to the output and it learn to reconstruct the input
-> `model.fit(X, X)`

Autoencoder

https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_autoencoder.ipynb

```
latent_dim = 9

autoencoder = Sequential([
    Flatten(name='encoder_flatten', input_shape=(28, 28)),
    Dense(300, activation='relu', name='encoder_1'),
    Dense(100, activation='relu', name='encoder_2'),
    Dense(latent_dim, activation='relu', name='encoder_output'),
    Dense(100, activation='relu', name='decoder_1'),
    Dense(300, activation='relu', name='decoder_2'),
    Dense(28*28, activation='relu', name='decoder_output'),
    Reshape((28, 28), name='decoder_reshape')
])

autoencoder.compile(optimizer='adam', loss=MeanSquaredError())
autoencoder.summary()
```

Encoding

Decoding

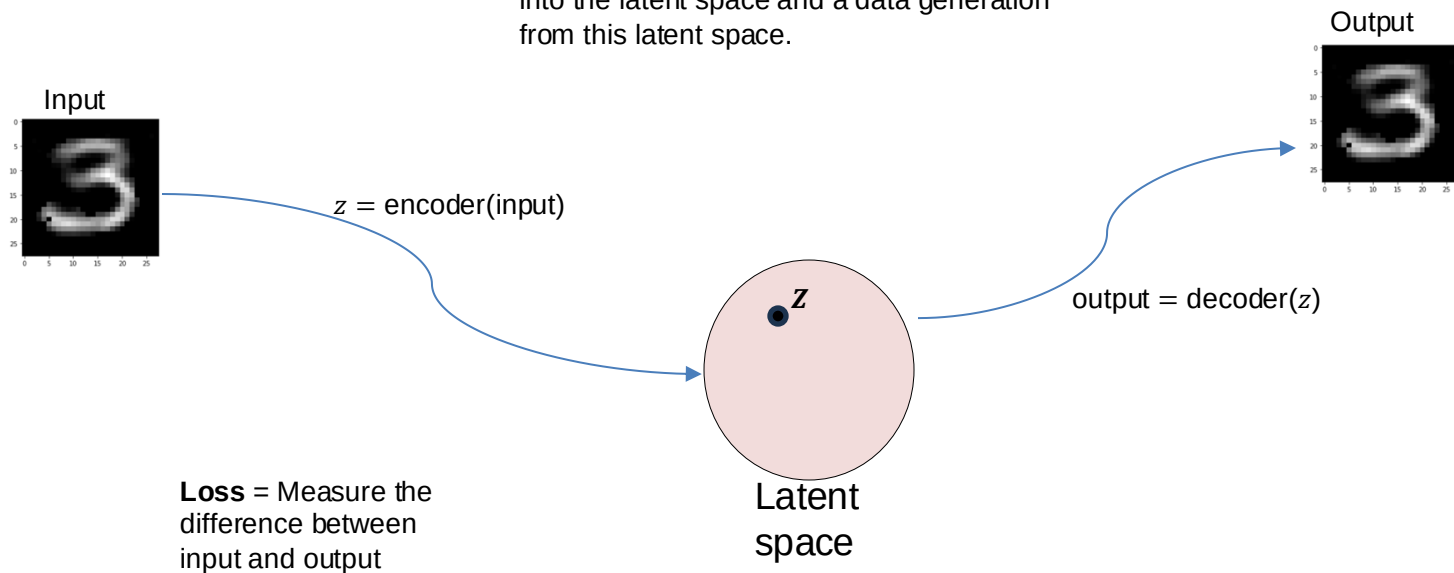
The loss of an autoencoder is called reconstruction loss, and can be defined simply as the squared error between the input and generated samples

Input/output are the same

```
autoencoder.fit(x_train, x_train,
               epochs=15,
               shuffle=True,
               validation_data=(x_test, x_test))
```

Autoencoder

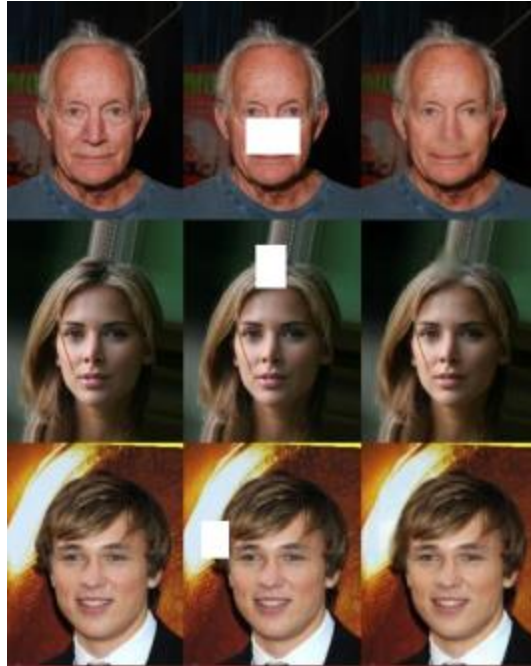
An autoencoder performs a direct projection into the latent space and a data generation from this latent space.



Autoencoder – applications

- Dimension reduction
- Classification (one-class per autoencoder)
- Anomaly detection
- Denoising
- ...
- (Synthetic data generation)

Denoising autoencoders

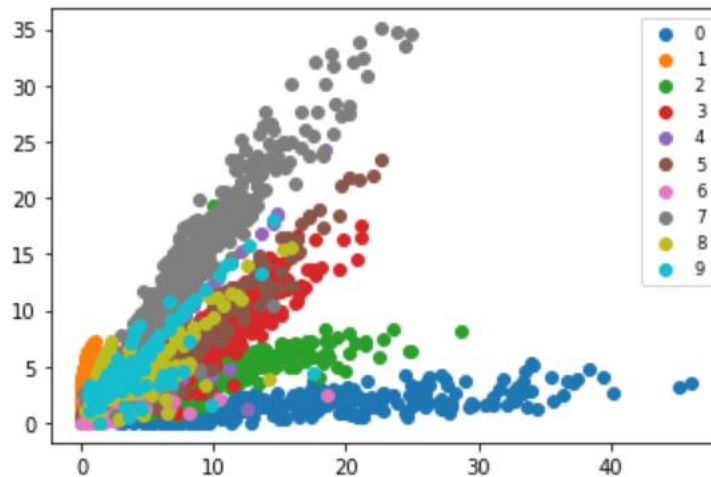


Latent space

Encoder map the (28*28) image to $\mathbb{R}^2$

Layer (type)	Output Shape	Param #
encoder_flatten (Flatten)	(None, 784)	0
encoder_1 (Dense)	(None, 300)	235500
encoder_2 (Dense)	(None, 100)	30100
encoder_output (Dense)	(None, 2)	202
decoder_1 (Dense)	(None, 100)	300
decoder_2 (Dense)	(None, 300)	30300
decoder_output (Dense)	(None, 784)	235984
decoder_reshape (Reshape)	(None, 28, 28)	0

=====
Total params: 532,386
Trainable params: 532,386
Non-trainable params: 0
=====



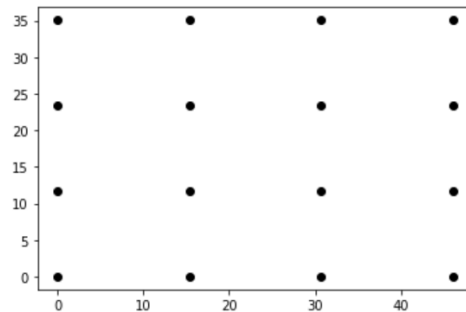
https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_autoencoder_laten_space.ipynb

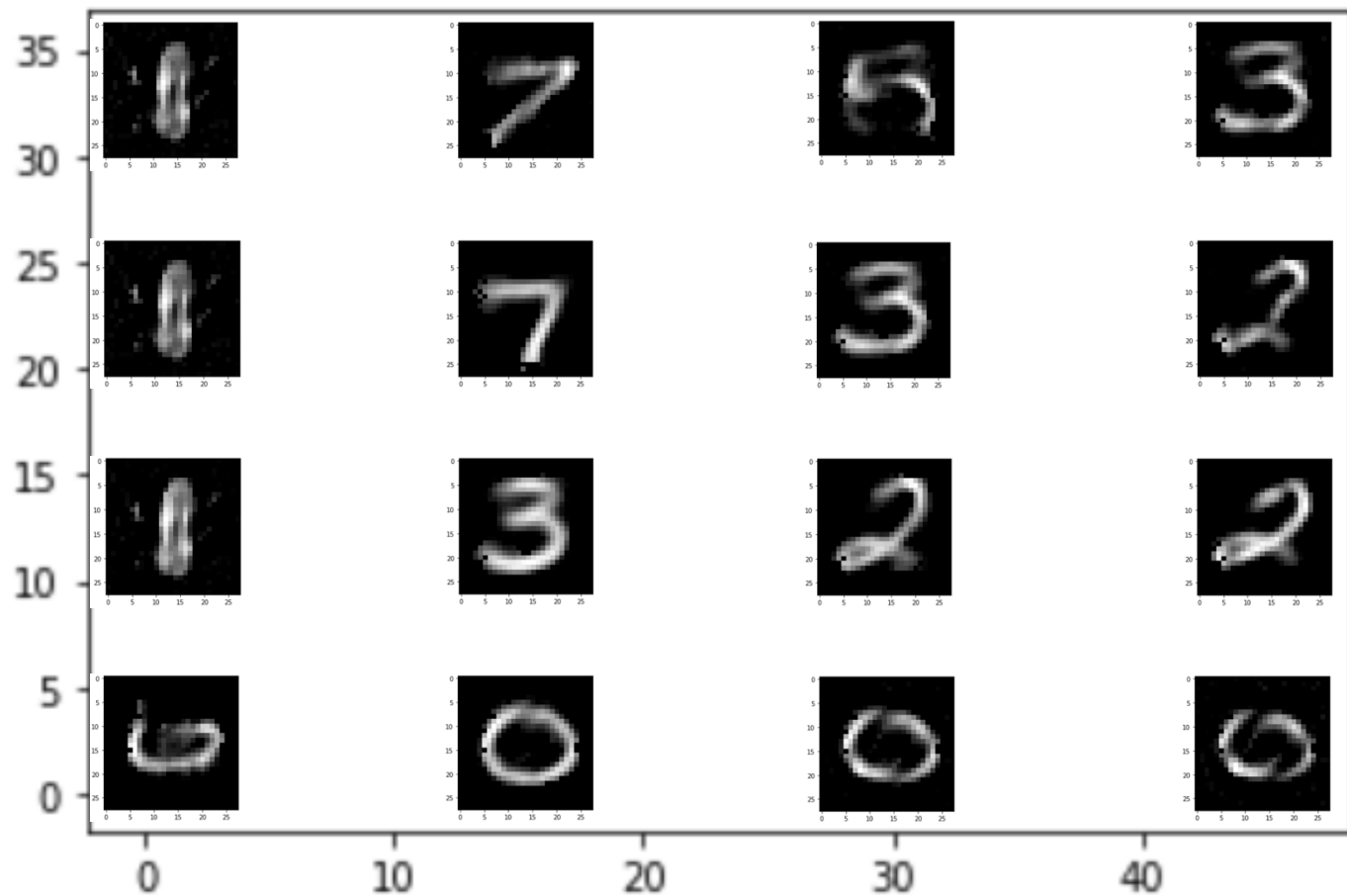
Latent space

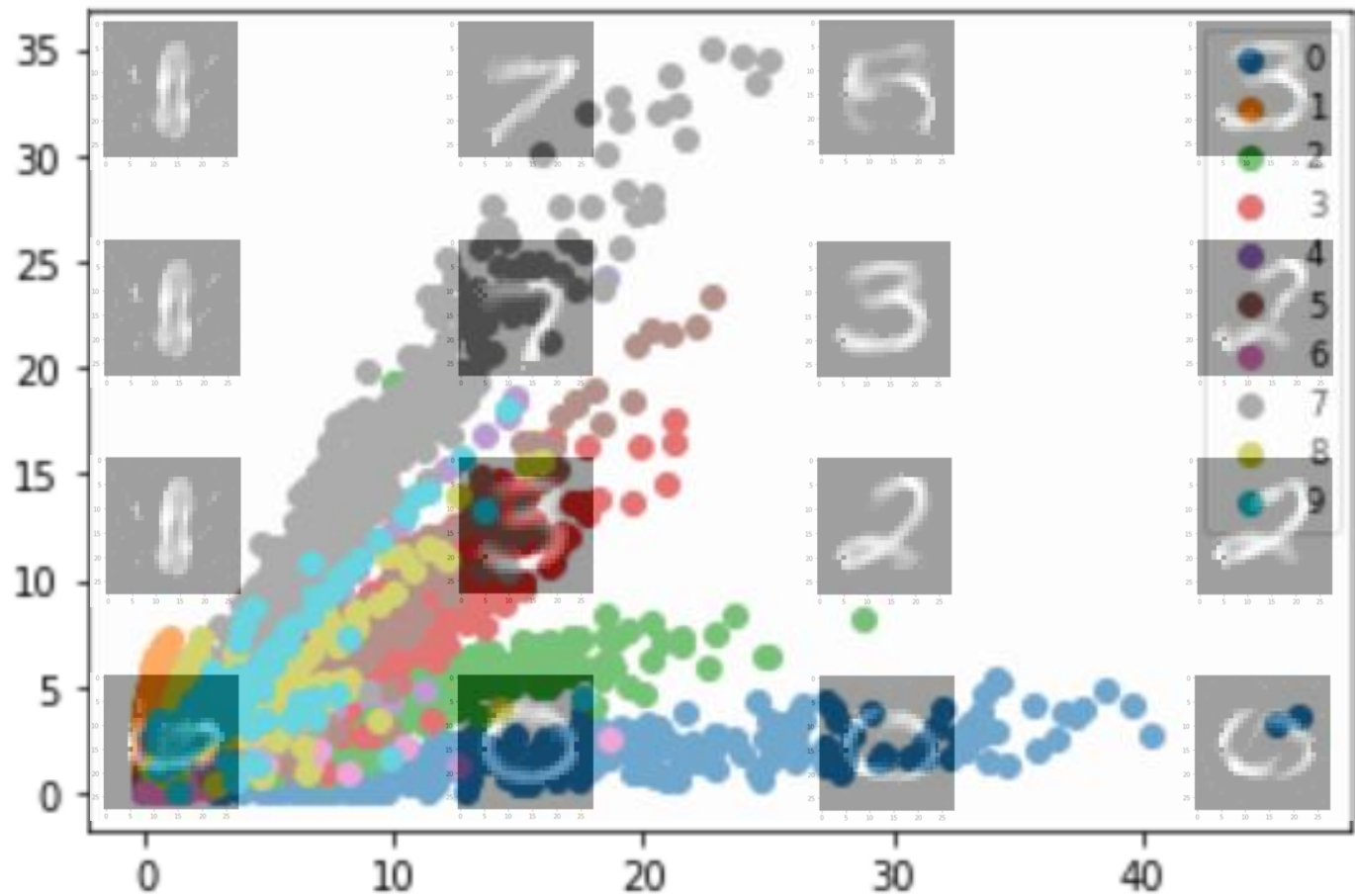
Layer (type)	Output Shape	Param #
encoder_flatten (Flatten)	(None, 784)	0
encoder_1 (Dense)	(None, 300)	235500
encoder_2 (Dense)	(None, 100)	30100
encoder_output (Dense)	(None, 2)	202
decoder_1 (Dense)	(None, 100)	300
decoder_2 (Dense)	(None, 300)	30300
decoder_output (Dense)	(None, 784)	235984
decoder_reshape (Reshape)	(None, 28, 28)	0
Total params: 532,386		
Trainable params: 532,386		
Non-trainable params: 0		

Decoder map an element for the $\mathbb{R}^2$
latent space to a (28*28) image!!

It is therefore possible to sample in the Latent space
to generate new images !!!
→ generative modeling



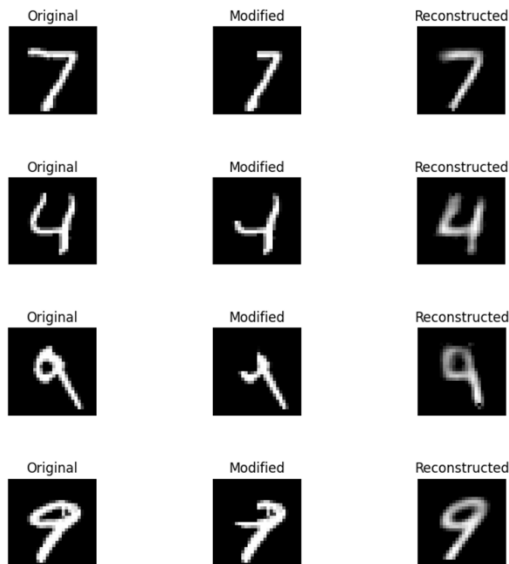




Reconstruction

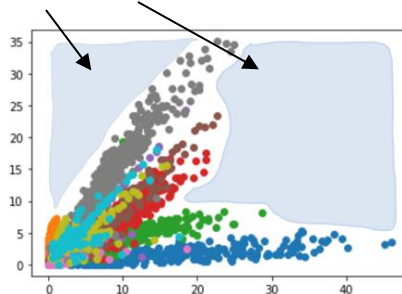
```
# Remove 25% of the image by replacing the upper left corner with black pixels  
modified_images = original_images.copy()  
corner_size = (int(0.5 * 28), int(0.5 * 28)) # Size of the upper left corner to be blackened  
modified_images[:, :corner_size[0], :corner_size[1]] = 0
```

```
# Autoencoder with latent_dim = 5  
reconstructed_images = autoencoder.predict(modified_images)
```



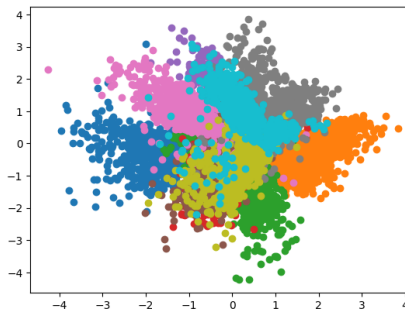
Variational autoencoders (VAEs)

- VAE are more accurate in generative tasks compared to traditional autoencoders.
- Classical autoencoders learn to compress and reconstruct data by minimizing the difference between input and output.
- But for generative tasks, samples are not well distributed



It makes no sense to take samples from an empty area to generate new images.

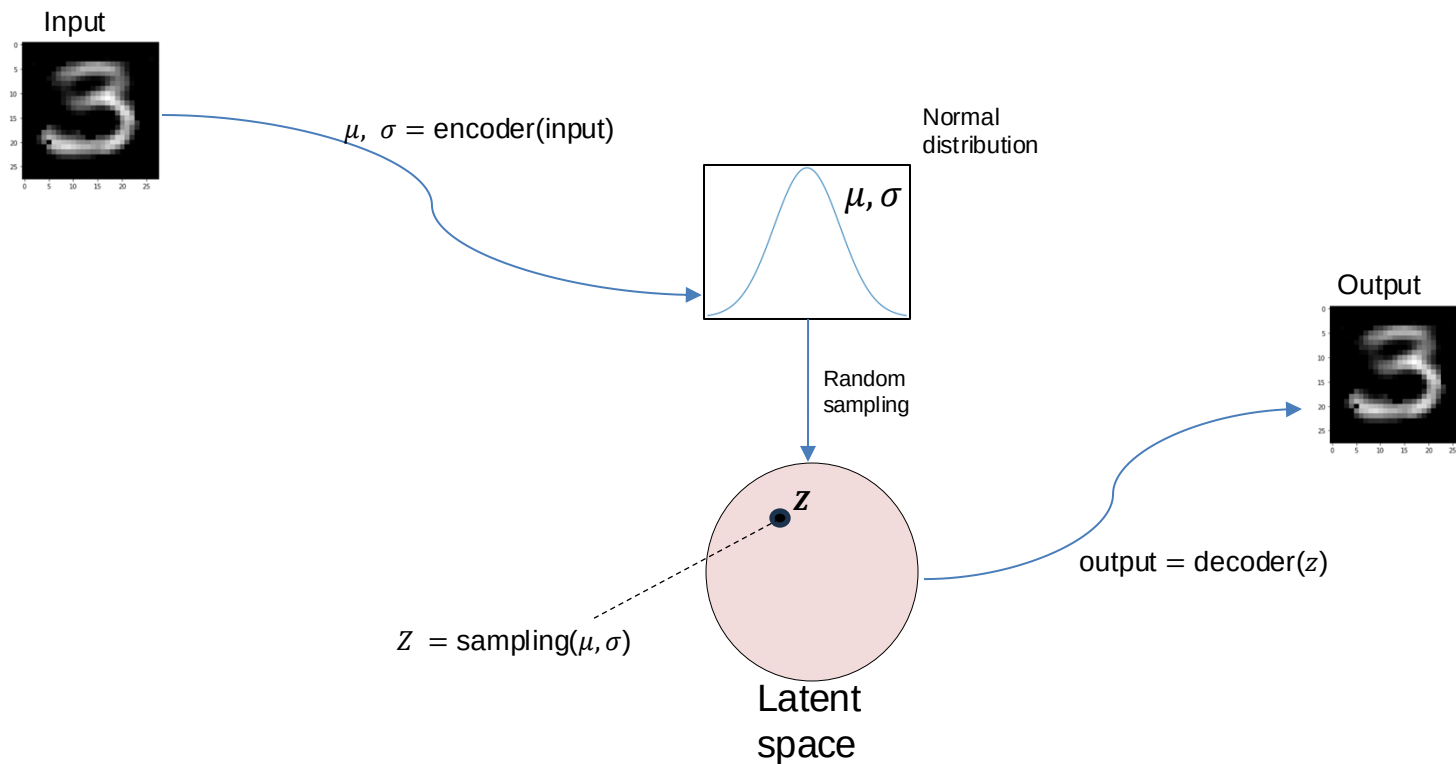
- VAEs, on the other hand, are better for generative tasks.



The latent space is now continuous.

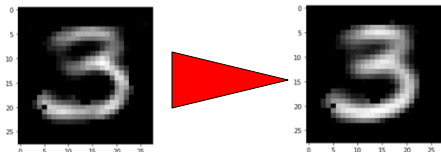
During training, the latent space is constrained to be similar as a Normal distribution.

Variational autoencoders (VAEs)



Variational autoencoders (VAEs) – loss function

Reconstruction loss:
Measure the difference
between input and output.



$loss_R$

Kullback-Leibler (KL) divergence:
Divergence between two Gaussian
distributions: $N(\mu, \sigma)$ vs. $N(0,1)$.

$loss_{KL}$



$$\text{Total Loss} = \alpha_1 loss_{KL} + \alpha_2 loss_R$$

The trick is to find a good
compromise between these
two component of the loss

KL divergence

The KL divergence is a measure of how one probability distribution Q differs from a second, reference probability distribution P .

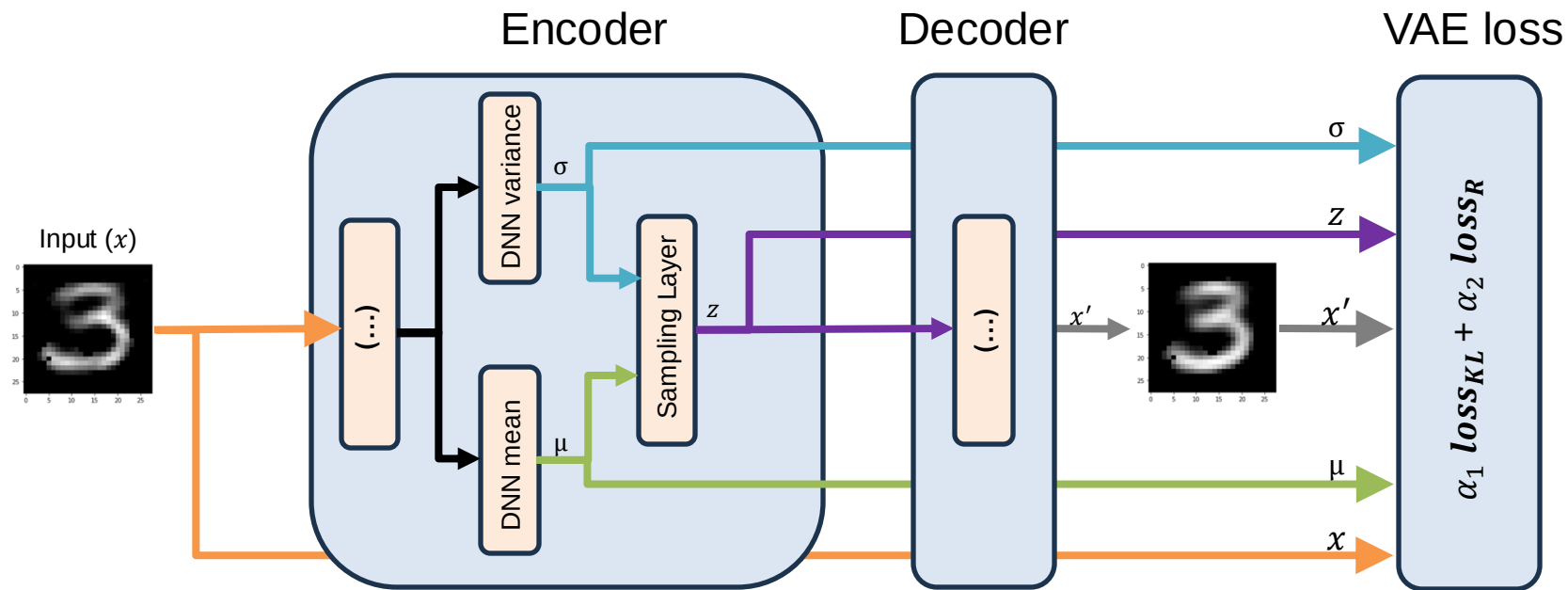
$$D_{KL}(P \parallel Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}$$

In the context of Variational Autoencoders (VAEs), we use the KL divergence to compare our learned distribution Q (a Gaussian with mean μ_i and standard deviation σ_i for all dimension i).) with a standard normal distribution P (Gaussian with mean 0 and standard deviation 1 for all dimension i).

$$\text{if } P = \mathcal{N}(0,1) \text{ and } Q = \mathcal{N}(\mu_i, \sigma_i), \quad \text{then } D_{KL}(P \parallel Q) = \sum_{i=1}^n (\sigma_i^2 + \mu_i^2 - \log(\sigma_i) - 1)$$

Mathematically, it helps us quantify the difference between these two distributions. A lower KL divergence indicates that our learned distribution is closer to the standard normal distribution.

Variational autoencoders (VAEs)



Example VAE – import & data (MNIST)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from tensorflow.keras import Model, datasets, Input
from tensorflow.keras.layers import Dense, Flatten, Reshape, Layer
from tensorflow.keras.losses import MeanSquaredError
from tensorflow.keras import backend as K
from tensorflow.keras.optimizers import Adam

# Load MNIST data
(x_train, _), (x_test, y_test) = datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
```

Example VAE – Encoder

```
latent_dim = 2
```

```
class Sampling(Layer):
```

```
    def call(self, inputs):
```

```
        z_mean, z_log_var = inputs
```

```
        batch = K.shape(z_mean)[0]
```

```
        dim = K.int_shape(z_mean)[1]
```

```
        epsilon = K.random_normal(shape=(batch, dim))
```

```
        return z_mean + K.exp(0.5 * z_log_var) * epsilon
```

```
# Encoder model
```

```
encoder_inputs = Input(shape=(28, 28), name='encoder_input')
```

```
x = Flatten(name='encoder_flatten')(encoder_inputs)
```

```
x = Dense(300, activation='relu', name='encoder_1')(x)
```

```
x = Dense(100, activation='relu', name='encoder_2')(x)
```

```
z_mean = Dense(latent_dim, name='z_mean')(x)
```

```
z_log_var = Dense(latent_dim, name='z_log_var')(x)
```

```
z = Sampling()(z_mean, z_log_var)
```

```
encoder = Model(encoder_inputs, [z_mean, z_log_var, z], name='encoder')
```

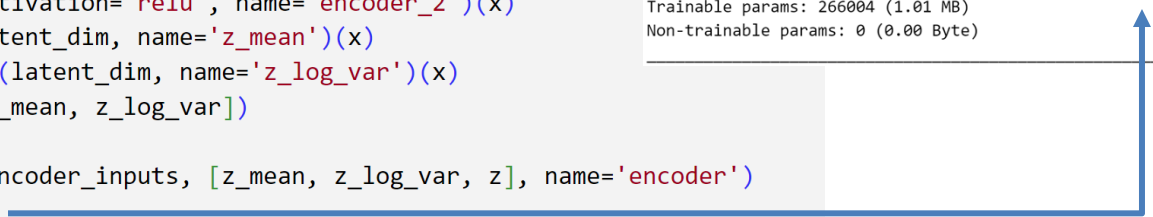
```
encoder.summary()
```

Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	[(None, 28, 28)]	0	[]
encoder_flatten (Flatten)	(None, 784)	0	['encoder_input[0][0]']
encoder_1 (Dense)	(None, 300)	235500	['encoder_flatten[0][0]']
encoder_2 (Dense)	(None, 100)	30100	['encoder_1[0][0]']
z_mean (Dense)	(None, 2)	202	['encoder_2[0][0]']
z_log_var (Dense)	(None, 2)	202	['encoder_2[0][0]']
sampling_2 (Sampling)	(None, 2)	0	['z_mean[0][0]', 'z_log_var[0][0]']

=====

Total params: 266004 (1.01 MB)
Trainable params: 266004 (1.01 MB)
Non-trainable params: 0 (0.00 Byte)

=====



Example VAE – Decoder

```
latent_inputs = Input(shape=(latent_dim,), name='z_sampling')
x = Dense(100, activation='relu', name='decoder_1')(latent_inputs)
x = Dense(300, activation='relu', name='decoder_2')(x)
x = Dense(28 * 28, activation='sigmoid', name='decoder_output')(x)
decoder_outputs = Reshape((28, 28), name='decoder_reshape')(x)

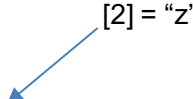
decoder = Model(latent_inputs, decoder_outputs, name='decoder')
decoder.summary()
```



Layer (type)	Output Shape	Param #
=====		
z_sampling (InputLayer)	[(None, 2)]	0
decoder_1 (Dense)	(None, 100)	300
decoder_2 (Dense)	(None, 300)	30300
decoder_output (Dense)	(None, 784)	235984
decoder_reshape (Reshape)	(None, 28, 28)	0
=====		
Total params: 266584 (1.02 MB)		
Trainable params: 266584 (1.02 MB)		
Non-trainable params: 0 (0.00 Byte)		

Example: VAE model = Encoder + Decoder

```
vae_outputs = decoder(encoder(encoder_inputs)[2])  
vae = Model(encoder_inputs, vae_outputs, name='vae')
```



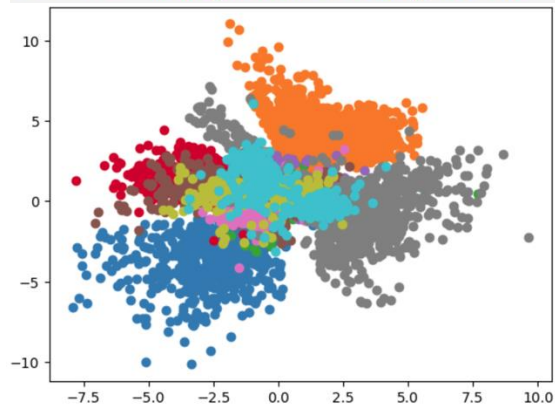
[2] = "z"

Loss function

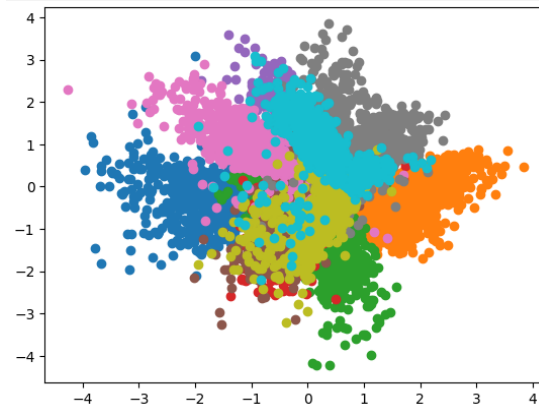
```
# Loss function  
a1 = 0.5 # weight for reconstruction loss  
a2 = 0.5 # weight for KL divergence loss  
  
reconstruction_loss = MeanSquaredError()(K.flatten(encoder_inputs), K.flatten(vae_outputs)) * 28 * 28  
  
kl_loss = K.exp(z_log_var) + K.square(z_mean) - z_log_var - 1  
kl_loss = K.sum(kl_loss, axis=-1)  
vae_loss = K.mean(a1 * reconstruction_loss + a2 * kl_loss)
```

Example VAE – Embeddings

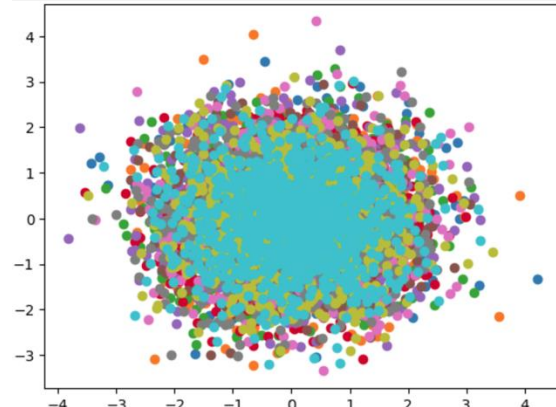
```
a1 = 0.9 # weight for reconstruction loss  
a2 = 0.1 # weight for KL divergence loss
```



```
a1 = 0.5 # weight for reconstruction loss  
a2 = 0.5 # weight for KL divergence loss
```



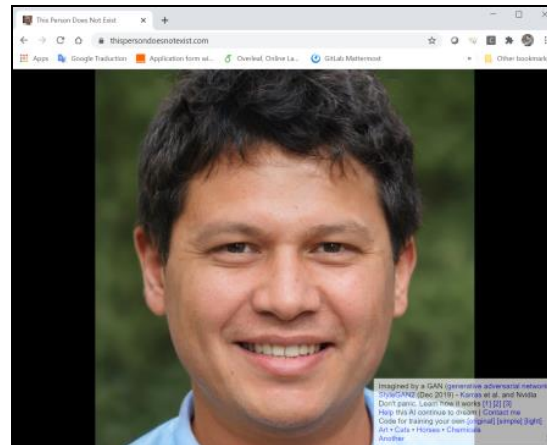
```
a1 = 0.1 # weight for reconstruction loss  
a2 = 0.9 # weight for KL divergence loss
```



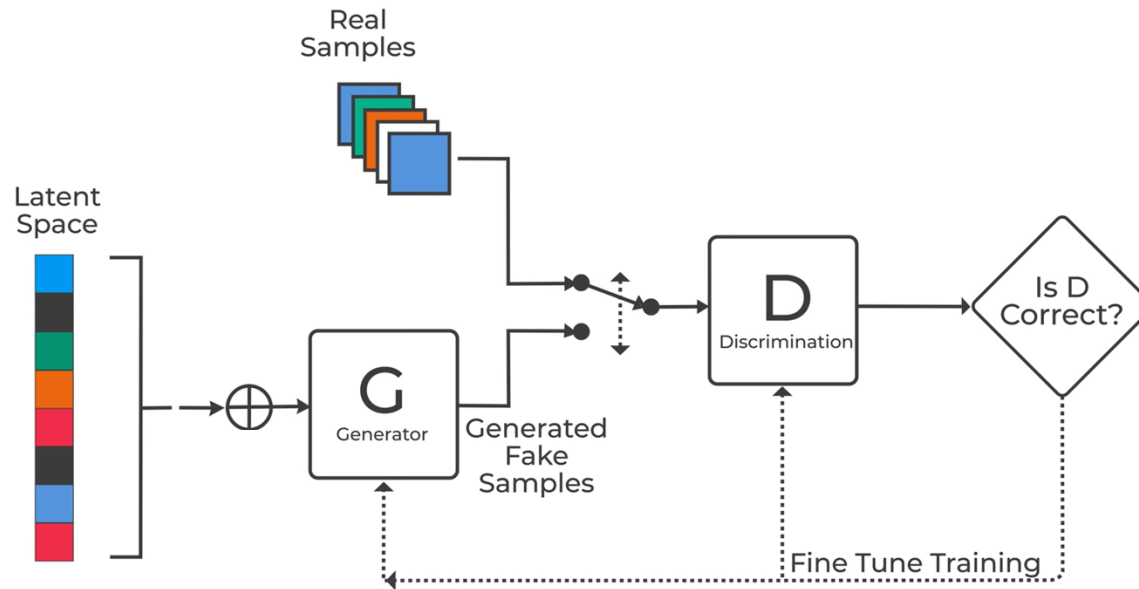
Generative Adversarial Networks (GAN)

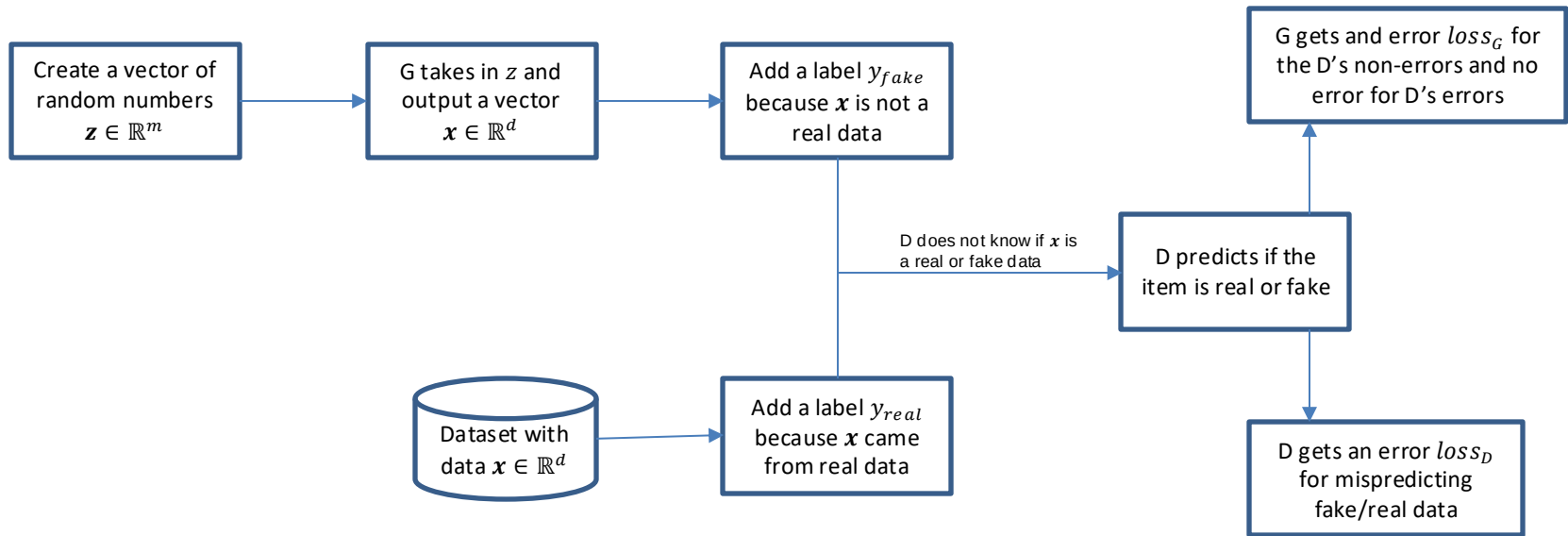
A GAN is an unsupervised method to generate new data

- It's a generative model where two networks are placed in competition
- The first network is the generator, it generates a sample (eg an image), while its opponent, the discriminator, tries to detect if a sample is real or if it is the result of the generator.
- Learning these networks is difficult in practice, with significant problems of non-convergence



<https://thispersondoesnotexist.com/>

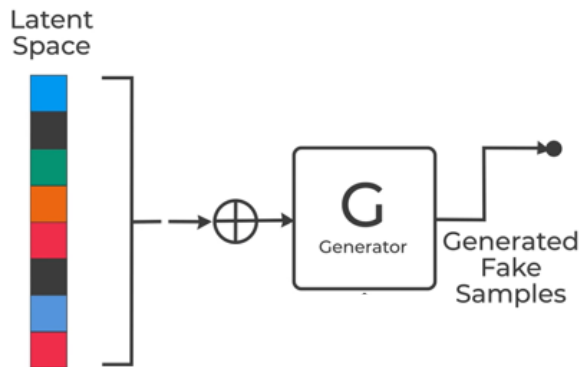




Latent space

- G works by taking from a *latent space* a vector $z \in \mathbb{R}^m$ and predict an output with the same shape as the real data.
- After learning, if we do a good job (convergence is difficult to achieve with GAN models), the generator G can use this latent representation z as a compact representation of the data – a kind of compression.
- In practice, we use a simple representation in which each latent variable is sampled from a Gaussian distribution $z_i \sim N(0,1)$.

During learning: it is up to the generator to learn how to make something meaningful out of these values.



Roughly speaking you can think of G as a sketch artist who must take in a description z and, from that, construct an accurate picture for the output.

Two loss functions

- D and G are both computing their loss from D's output.
- The loss of G is somehow the opposite of the loss of D.
- Let $J(\cdot, \cdot)$ be our classification loss (e.g., binary cross entropy)
- There are 2 models (G and D) and two data sources (y_{real} and y_{fake}) $\rightarrow$ giving the 4 combinations
 - The loss of D on real data: $J(D(x), y_{real})$
 - The loss of D on fake data: $J(D(G(z)), y_{fake})$
 - The loss of G on real data: G does not care what D says about real data $\rightarrow$ nothing happens to G when real data is used
 - The loss of G on fake data: $J(D(G(z)), \underline{y_{real}})$

G wants D to call its (by error) 'real data', because that means it has successfully tricked D

	Real data x		Fake data z	
$loss_D$	$J(D(x), y_{real})$	+	$J(D(G(z)), y_{fake})$	
$loss_G$	0	+	$J(D(G(z)), y_{real})$	

$\longrightarrow$

$$\begin{aligned} loss_D &= J(D(x), y_{real}) + J(D(G(z)), y_{fake}) \\ loss_G &= J(D(G(z)), y_{real}) \end{aligned}$$

Gan -- code

https://github.com/ocaelen/intro_deeplearning/blob/main/gan.ipynb

```
batch_size = 128
latent_d = 128
neurons = 512
out_shape = (-1, 28, 28)
num_epochs = 10

def fcLayer(in_neurons, out_neurons, leak=0.1):
    return nn.Sequential(
        nn.Linear(in_neurons, out_neurons),
        nn.LeakyReLU(leak),
        nn.LayerNorm(out_neurons)
    )
```

- in_neurons: how many inputs to this layer
- out_neurons: how many outputs for this layer
- leak: the leaky relu leak value.

```
def simpleGAN(latent_d, neurons, out_shape, sigmoidG=False, leak=0.2):
    G = nn.Sequential(
        fcLayer(latent_d, neurons, leak),
        fcLayer(neurons, neurons, leak),
        fcLayer(neurons, neurons, leak),
        fcLayer(neurons, neurons, leak),
        nn.Linear(neurons, abs(np.prod(out_shape))),
        View(out_shape)
    )

    if sigmoidG:
        G = nn.Sequential(G, nn.Sigmoid())

    D = nn.Sequential(
        nn.Flatten(),
        fcLayer(abs(np.prod(out_shape)), neurons, leak),
        fcLayer(neurons, neurons, leak),
        fcLayer(neurons, neurons, leak),
        fcLayer(neurons, neurons, leak),
        nn.Linear(neurons, 1 )
    )

    return G, D
G, D = simpleGAN(latent_d, neurons, out_shape, sigmoidG=True)
```

This function will create a simple GAN for us to train. It will return a tuple (G, D), holding the generator and discriminator network, respectively.

- latent_d: the number of latent variables we will use as input to the generator G.
- neurons: how many hidden neurons to use in each hidden layer
- out_shape: the shape of the output of the discriminator D. This should be the same shape as the real data.
- sigmoidG: true if the generator G should end with a sigmoid activation, or False if it should just return unbounded activations

Gan -- code

```
G.to(device)
D.to(device)

loss_func = nn.BCEWithLogitsLoss()

real_label = 1
fake_label = 0

optimizerD = torch.optim.AdamW(D.parameters(), lr=0.0001, betas=(0.0, 0.9))
optimizerG = torch.optim.AdamW(G.parameters(), lr=0.0001, betas=(0.0, 0.9))
```

- Move models G and D to cpu or gpu
- Initialize BCEWithLogitsLoss function. The BCE loss is for binary classification problems, which ours is (real vs fake)
- Establish convention for real and fake labels during training
- Setup Adam optimizers for both G and D

Load the data

```
train_data = torchvision.datasets.MNIST("./", train=True, transform=transforms.ToTensor(), download=True)
test_data = torchvision.datasets.MNIST("./", train=False, transform=transforms.ToTensor(), download=True)

train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True, drop_last=True)
test_loader = DataLoader(test_data, batch_size=batch_size)
```

Gan -- code

```
G_losses = []
D_losses = []

for epoch in range(num_epochs):
    print(epoch)
    for data, class_label in train_loader:

        real_data = data.to(device)
        y_real = torch.full((batch_size,1), real_label, dtype=torch.float32, device=device)
        y_fake = torch.full((batch_size,1), fake_label, dtype=torch.float32, device=device)

        D.zero_grad()

        errD_real = loss_func(D(real_data), y_real)
        errD_real.backward()

        z = torch.randn(batch_size, latent_d, device=device)
        fake = G(z)
        errD_fake = loss_func(D(fake.detach()), y_fake)
        errD_fake.backward()

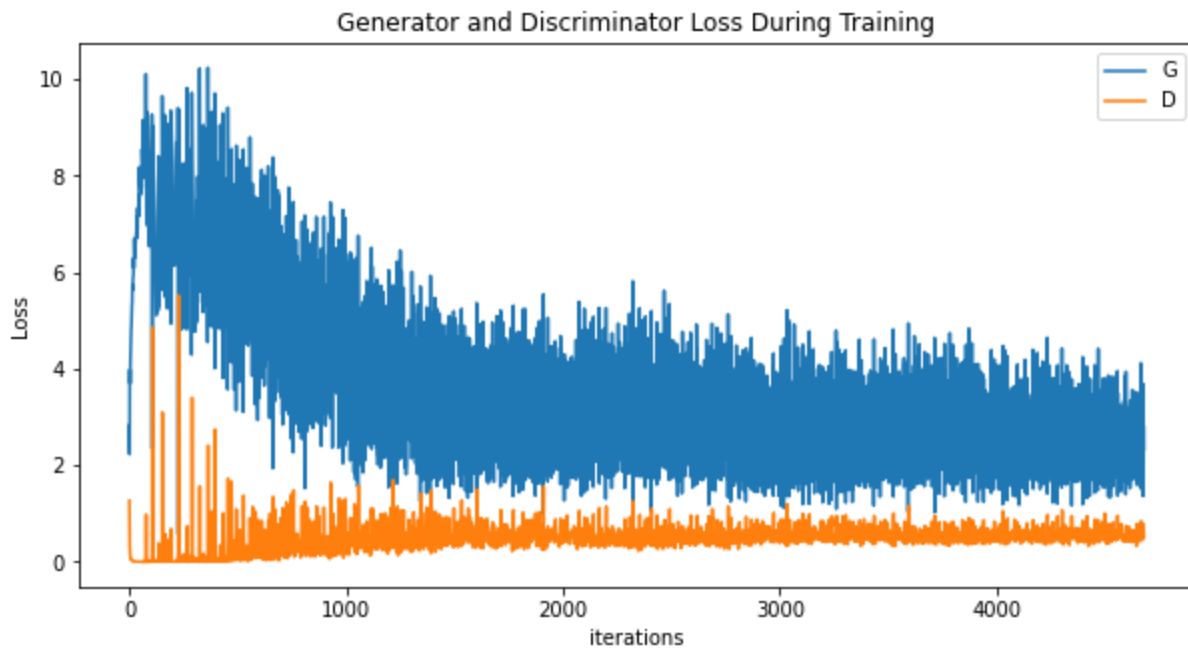
        errD = errD_real + errD_fake
        optimizerD.step()

        G.zero_grad()

        errG = loss_func(D(fake), y_real)
        errG.backward()
        optimizerG.step()

    G_losses.append(errG.item())
    D_losses.append(errD.item())
```

- prep batch & make labels
- Start update D
- Generate batch of latent vectors $z \sim N(\mathbf{0}, I)$
- Start update G



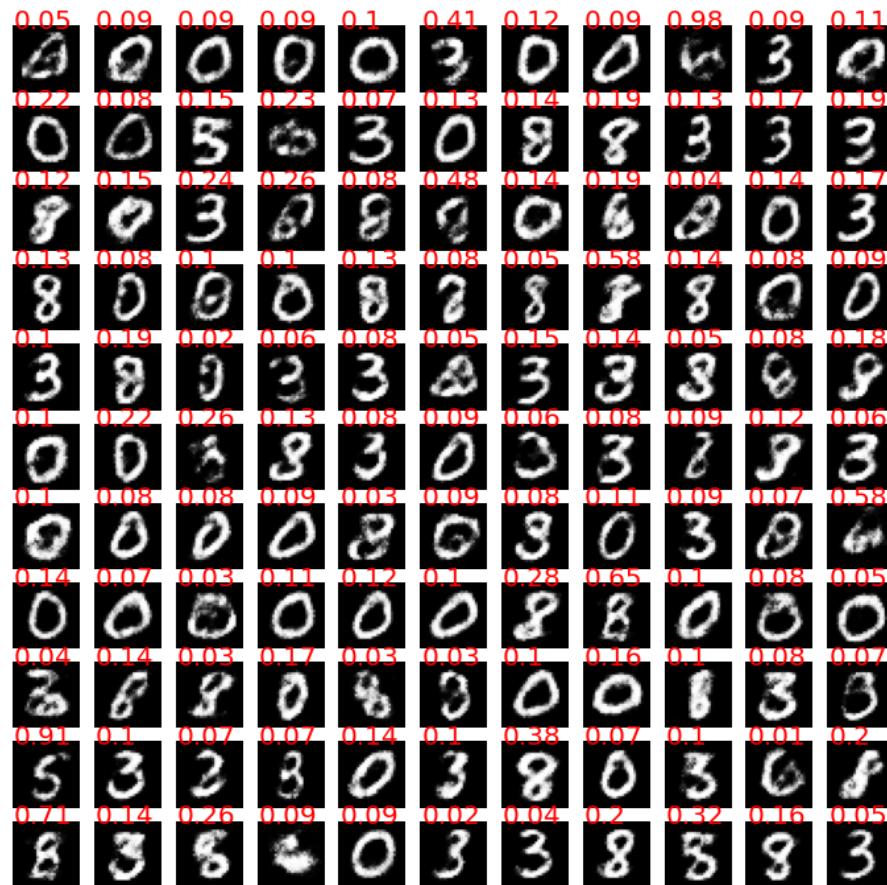
The D task seems to be easier than the G task

Gan -- code

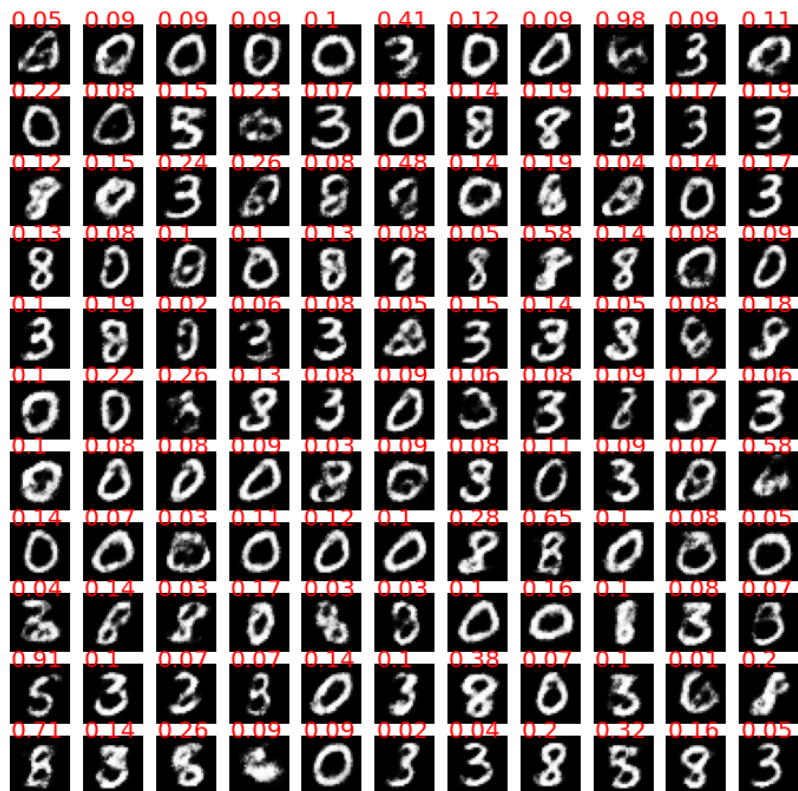
```
with torch.no_grad():  
    noise = torch.randn(batch_size, latent_d, device=device)  
    fake_digits = G(noise)  
    scores = torch.sigmoid(D(fake_digits))  
  
    fake_digits = fake_digits.cpu()  
    scores = scores.cpu().numpy().flatten()
```

this_digits_does_not_exist ;-)

To generate better images, we should use convolutional GAN
(not cover in this lecture)



Question for you:



The distribution of the numbers generated is not uniform (e.g; lot of “0” and few “4”)... why?



Mode collapse

Natural Language Processing (NLP)

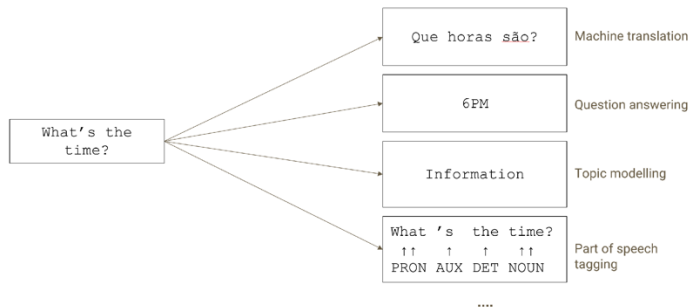
Natural Language Processing (NLP)

- NLP is rapidly growing in the industry
 - Dialogue:** ChatBots, Customer Service, Virtual Companion, Sentiment analysis
 - HealthCare:** understanding patient's health data and biomedical literature
 - ...



<https://techcrunch.com/2019/09/25/everything-amazon-announced-at-its-alexa-event-today/?qucounter=1>

- What is NLP?



- Fundamental concepts
 - Vocabulary:** A list of all acceptable words
 - Document:** A single, contiguous piece of text. Can be a tweet, an essay, or a book.
 - Corpus (plural corpora):** A collection of documents. E.g. the panama papers
 - Tokens:** E.g. an instance of a word in a piece of text
 - Syntax vs. Semantics**
 - Syntax:** The grammatical structure of language
 - Semantics:** The meaning of language

Pre-processing techniques (Tokenization)

We need to represent sentences in a way that a computer can handle

- Tokenization:
 - “We are studying tonight.” → [“we”, “are”, “studying”, “tonight”, “.”]
 - **Tokens** are independent and minimal textual components that have some definitive syntax and semantics
 - Word boundary:
 - Be careful, e.g., **U.S.A** is more accurate to be [“USA”] than [“U”, “S”, “A”]
 - Linguistic (example of a tool: <http://nlp.stanford.edu:8080/parser/index.jsp>)
 - don’t → [“do”, “n’t”]
 - that’s → [“that”, “s”]
 - Named Entity Recognition: In a text document there are particular terms that represent entities that
 - are more informative
 - have a unique context compared to the rest of the text
 - Examples of named entities :

Named Entity Type	Examples
PERSON	President Obama, Franz Beckenbauer
ORGANIZATION	WHO, ISRO, FC Bayern
LOCATION	Germany, India, USA, Mt. Everest
DATE	December, 2016-12-25
TIME	12:30:00 AM, one thirty pm
MONEY	Twenty dollars, Rs. 50, 100 GBP
PERCENT	20%, forty five percent
FACILITY	Stonehenge, Taj Mahal, Washington Monument
GPE	Asia, Europe, Germany, North America

- Comparison between different Tokenizer

	word_tokenize	WordPunctTokenizer	RegexTokenizer for matching	RegexTokenizer for white space	TweetTokenizer
0	[https, ., //t.co/9z2J3P33Uc]	[https, ., //, t, ., co, ., 9z2J3P33Uc]	[https, t, co, 9z2J3P33Uc]	[https://t.co/9z2J3P33Uc]	[https://t.co/9z2J3P33Uc]
1	[laugh/cry]	[laugh, ., cry]	[laugh, cry]	[laugh/cry]	[laugh, ., cry]
2	[😂😂😂😂😂😂]	[😂😂😂😂😂😂]	[]	[😂😂😂😂😂😂]	[😂, 😂, 😂, 😂, 😂, 😂]
3	[world, 's, problems]	[world, ' , s, problems]	[world's, problems]	[world's, problems]	[world's, problems]
4	[@, datagenera]	[@, datagenera]	[datagenera]	[@datagenera]	[@datagenera]
5	[It, 's, interesting]	[It, ' , s, interesting]	[It's, interesting]	[It's, interesting]	[It's, interesting]
6	[do, n't, spell, my, name, right]	[don, ' , t, spell, my, name, right]	[don't, spell, my, name, right]	[don't, spell, my, name, right]	[don't, spell, my, name, right]
7	[all-nighter]	[all, -, nighter]	[all, nighter]	[all-nighter]	[all-nighter]

Pre-processing techniques (Normalization)

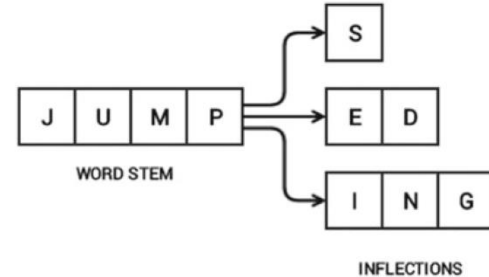
Case conversions, stopwords, repeating characters,...

- Remove stopwords
 - Stopwords are words that have little or no significance to most text mining tasks.
 - usually refers to most common words e.g. and, as, be, could, why, who ...
 - Usually, a precompiled list of about 300-400 words (English)
 - Not all stopwords lists contain exactly the same words
 - Stopwords lists are language dependent
 - Have a look at stopword lists for more than 40 languages and more than one English list at the link below
<https://www.ranks.nl/stopwords>
- Case conversions:
 - We might want to modify the case of words or sentences to make processing easier computationally WORD $\neq$ word
- Correct repeating characters
 - Use a combination of syntax and semantics to correct incorrectly spelled words
e.g. finallllyyyy $\rightarrow$ finaly
 - Regex patterns are often used to resolve this issue or Spell checker

Pre-processing techniques (Normalization)

Stemming & Lemmatization

- Stemming
 - Extract a base or root form of a word
 - does not need to be identical to the morphological root of the word
 - a word stem does not need to be a word !
 - There are different stemmers implementations
 - Snowball, Lovins, Porter ...
 - Eg,
 - argue or arguing -> argu
 - zombies -> zombi
 - having -> hav
 - stemming -> stem
- Lemmatization
 - process of grouping together inflected forms of a word



word	lemma
argue / arguing	argue
zombies	zombie
having	have
stemming	stem
sung	sing

Bag of Words

- One of the simplest and powerful technique to extract features from text documents
- Convert text documents into vectors
 - Each vector represents the frequency of all the **distinct words** that are present in the document vector space (for that specific document)

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

Python

```
[1]: from sklearn.preprocessing import MultilabelBinarizer  
import pandas as pd
```

```
[2]: docs = ['I love machine learning',  
            'A machine can love',  
            'I love love love']  
  
tokens_docs = [doc.split(" ") for doc in docs]  
tokens_docs
```

```
[2]: [['I', 'love', 'machine', 'learning'],  
      ['A', 'machine', 'can', 'love'],  
      ['I', 'love', 'love', 'love']]
```

```
[3]: mlb = MultilabelBinarizer()  
pd.DataFrame(mlb.fit_transform(tokens_docs), columns=mlb.classes_)
```

```
[3]:
```

	A	I	can	learning	love	machine
0	0	1	0	1	1	1
1	1	0	1	0	1	1
2	0	1	0	0	1	0

TF-IDF

- If I have a corpus of documents, how important is a given word for any of the document that I have in my corpus?
- Tf-Idf is a multiplication of two metrics:
 - Tf (Term Frequency)
 - Idf (Inverse Document Frequency).

- Term Frequency of a given word t in a document d is:

$$tf(t, d) = \frac{\# \text{ occurrence of } t \text{ in } d}{\# \text{ total words in } d}$$

- It is an intuitive measure of how often does a word occur in each document.
 - But it is not sufficient to use alone in solving our original problem because certain words are going occur a lot in all of the documents.
 - For example : 'The', 'and', ...
 - Therefore, tf alone is not able to identify the special words in each documents

- Inverse Document Frequency is function of a word t and a corpus of documents D :

$$idf(t, D) = \log \frac{\# \text{ number of document in } D}{\# \text{ number of document with } t}$$

- $tf-idf(t, d, D) = tf(t, d) * idf(t, D)$
- Question for you:
 - Why the word 'the' will have a small $tf-idf$ value?
 - What does you need to take to have high value?

- In general, tf-idf works better than bag-of-words for text preprocessing.

Embedding & word2vec (Text processing)

Categorical Variables & One-Hot Encoding

- Lot of learning machine algorithms are not able to deal directly with input categorical variables.
- One-Hot Encoding is a classic method for handling categorical variables.

Example: Merchant Category Codes

- Supermarkets
- Grocery Stores
- Bakeries
- Hospitals
- Tourist Attractions
- ...

One-hot encoding

Categorical Variables	One-Hot Encoding
Supermarkets	1 0 0 0 0 0 0 ... 0 0
Grocery Stores	0 1 0 0 0 0 0 ... 0 0
Bakeries	0 0 1 0 0 0 0 ... 0 0
Hospitals	0 0 0 1 0 0 0 ... 0 0
Tourist Attractions	0 0 0 0 1 0 0 ... 0 0
...	...

Afterwards, the **categorical variable** is **replaced** in the training and scoring sets **by the corresponding one-hot vector**.

Drawbacks:

- The representation **size grows** with the corpus
- Each vector is **equidistant** from every other vector
- ...

Python

```
[1]: import string
import numpy as np
from keras.preprocessing.text import Tokenizer
import itertools

[2]: docs = ['I love love machine learning',
            'A machine can love',
            'I love love love']

tokens_docs = [doc.split(" ") for doc in docs]
tokens_docs

[2]: [['I', 'love', 'love', 'machine', 'learning'],
      ['A', 'machine', 'can', 'love'],
      ['I', 'love', 'love', 'love']]

[3]: # First, build an index of all tokens in the data.
all_tokens = itertools.chain.from_iterable(tokens_docs)
word_to_id = {token: idx for idx, token in enumerate(set(all_tokens))}
word_to_id

[3]: {'love': 0, 'machine': 1, 'can': 2, 'I': 3, 'A': 4, 'learning': 5}
```

```
[4]: nb_words = len(word_to_id)

res = []
for tokens_doc in tokens_docs:
    mat = np.zeros((len(tokens_doc), nb_words))
    for j, word in list(enumerate(tokens_doc)):
        index = word_to_id.get(word)
        mat[j, index] = 1.
    res.append(mat)

[5]: for i, x in enumerate(res):
    print(docs[i])
    print(x)
    print()
```

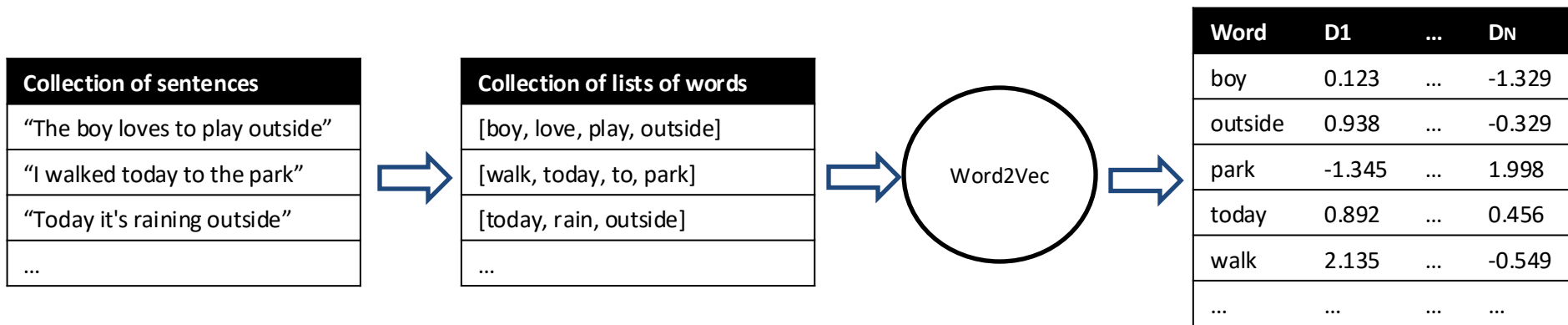
```
I love love machine learning
[[0. 0. 0. 1. 0. 0.]
 [1. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1.]]
```

```
A machine can love
[[0. 0. 0. 0. 1. 0.]
 [0. 1. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0.]]
```

```
I love love love
[[0. 0. 0. 1. 0. 0.]
 [1. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0.]]
```

Set of words & Word2Vec

- Word2Vec is often used to learn linguistic regularities by representing words with vectors.
- The ordinary usage is to feed it with sentences from a corpus of documents and to collect the embedding vectors.



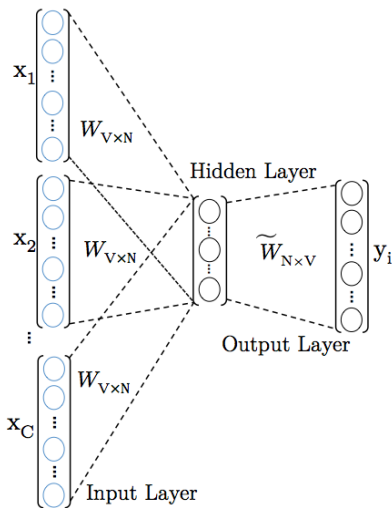
Word2Vec (CBOW)

Given a context window of words, we try to predict the appropriate word

[boy, love, play, outside]

context [one before & after]	to predict
[], [love]	boy
[boy], [play]	love
[love], [outside]	play
[play], []	outside
...	...

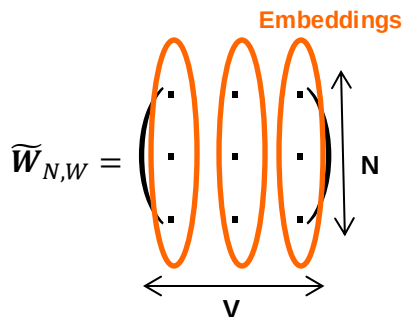
As training set



- The neurons colored in blue are 0 with a unique 1 per block because the **input words** are **one-hot encoded**.
- There is a constraint so that the input weighted matrices are identical.
- After a sufficient number of training iterations over the corpus, the **embedding vectors** are available in $\tilde{W}$.
- **C**: Number of “contextual” words
- **N**: the dimension of the embedding vectors
- **V**: the cardinal of the vocabulary (i.e. the different words in the corpus)
- $W, \tilde{W}$: the weighted matrices of the NN

Word2Vec (CBOW)

After a sufficient number of training iterations over the corpus, the **embedding vectors** are available in $\widetilde{W}$.



Word	D1	...	DN
boy	0.123	...	-1.329
outside	0.938	...	-0.329
park	-1.345	...	1.998
today	0.892	...	0.456
walk	2.135	...	-0.549
...	...	...	...

The main parameters of the Word2Vec:

C (size of the context) : 5

N (the dimension of the embedding vectors)



Figure 3: A word2vec embedding of tokens from C/C++ source code

Credit: *Automated software vulnerability detection with machine learning* J.A. Harer et al.

Recurrent Neural Networks (Deep sequence modeling)

Motivations: Handle sequential data

- Predict value of an output variable given sequential past observations.

Text data: 

Signal data: 

- Applications: Text translation, time series prediction, speech recognition, control, customer behavior, ...

Notation

$$\mathbf{x}^1 = (\text{He} \text{ Came } \text{to} \text{ dinner } \text{with} \text{ us})^T = (x^1[1], x^1[2], \dots, x^1[t], \dots, x^1[\mathcal{T}_x^1])^T \text{ where } \mathcal{T}_x^1 = 6$$

$$\mathbf{x}^2 = (\text{I} \text{ love } \text{learning})^T = (x^2[1], x^2[2], x^2[\mathcal{T}_x^2 = 3])^T$$

$\mathcal{T}_x^i$ is the length of the input sequence number i where $1 \leq i \leq N$.
Note that sequence length can be different for each observation.

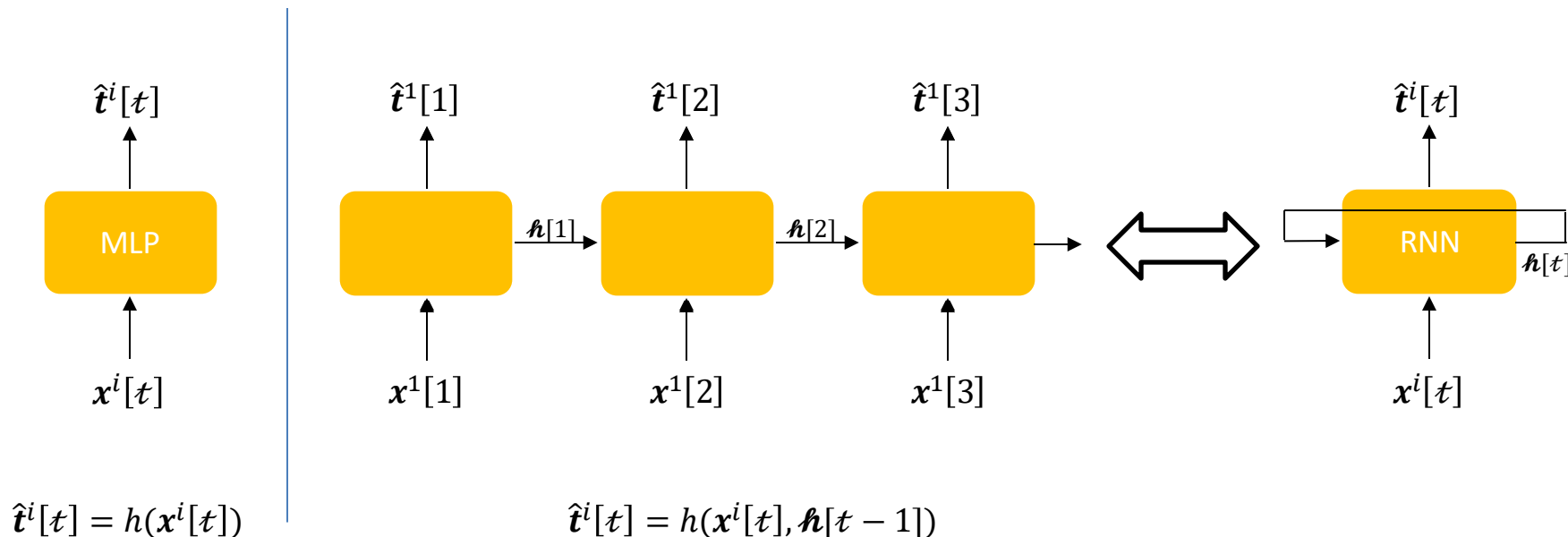
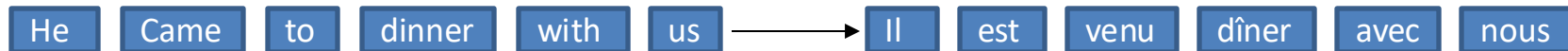
t is used as index in the sequences.

We have $x^i[t] \in \mathbb{R}^n$ for all i and t .
(Which is not the case in the above example! $\rightarrow$ e.g., use word2vec)

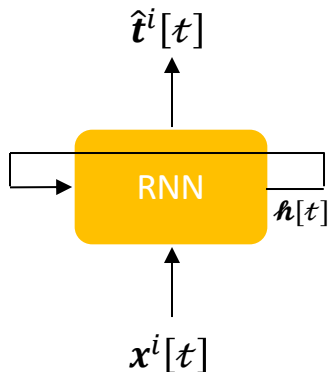
$\mathbf{t}^i$ is the corresponding target of $\mathbf{x}^i$ where $\mathbf{t}^i = (t^i[1], \dots, t^i[\mathcal{T}_t^i])^T$

Sequence Modeling & Simple Recurrent Networks (SRN)

Assume the following applications : text translation



Simple Recurrent Networks – also known as Elman network



- The RNN has an internal hidden state $\mathbf{h}[t]$.
- This state plays the role of a short-term memory.
- $\mathbf{h}[t]$ is updated at each time step t .
- $\mathbf{h} \in \mathbb{R}^{n_1 \times 1}$ where n_1 is the number of hidden neurons.
- Initial $\mathbf{h}[0]$: Either random, constant, or from a known past state

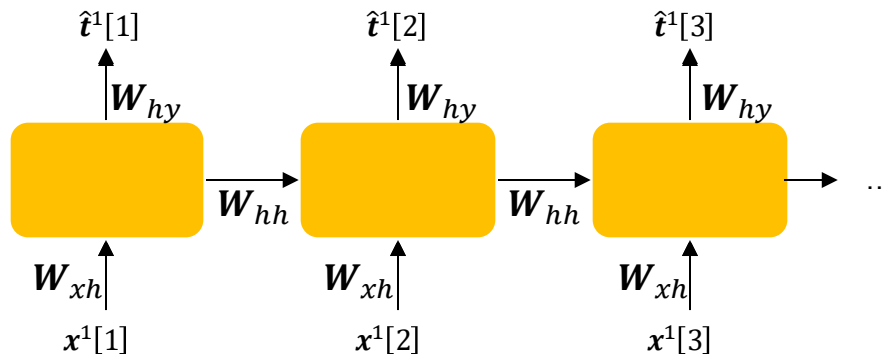
Update hidden state:

$$\mathbf{h}[t] = \tanh(\mathbf{W}_{hh} \mathbf{h}[t-1] + \mathbf{W}_{xh} \tilde{\mathbf{x}}^i[t] + \text{bias})$$

Output (given the context, many activation functions are possible):

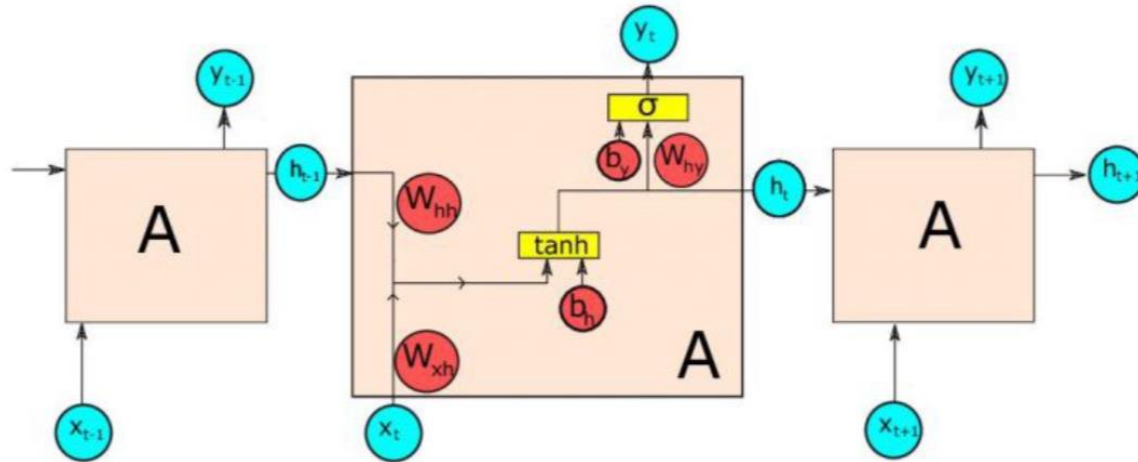
$$\hat{\mathbf{t}}^i[t] = \mathbf{W}_{hy} \mathbf{h}[t]$$

$$\begin{aligned} \mathbf{W}_{hh} &\in \mathbb{R}^{n_1 \times n_1} \\ \mathbf{W}_{xh} &\in \mathbb{R}^{n_1 \times (n+1)} \\ \mathbf{W}_{hy} &\in \mathbb{R}^{M \times (n_1+1)} \end{aligned}$$



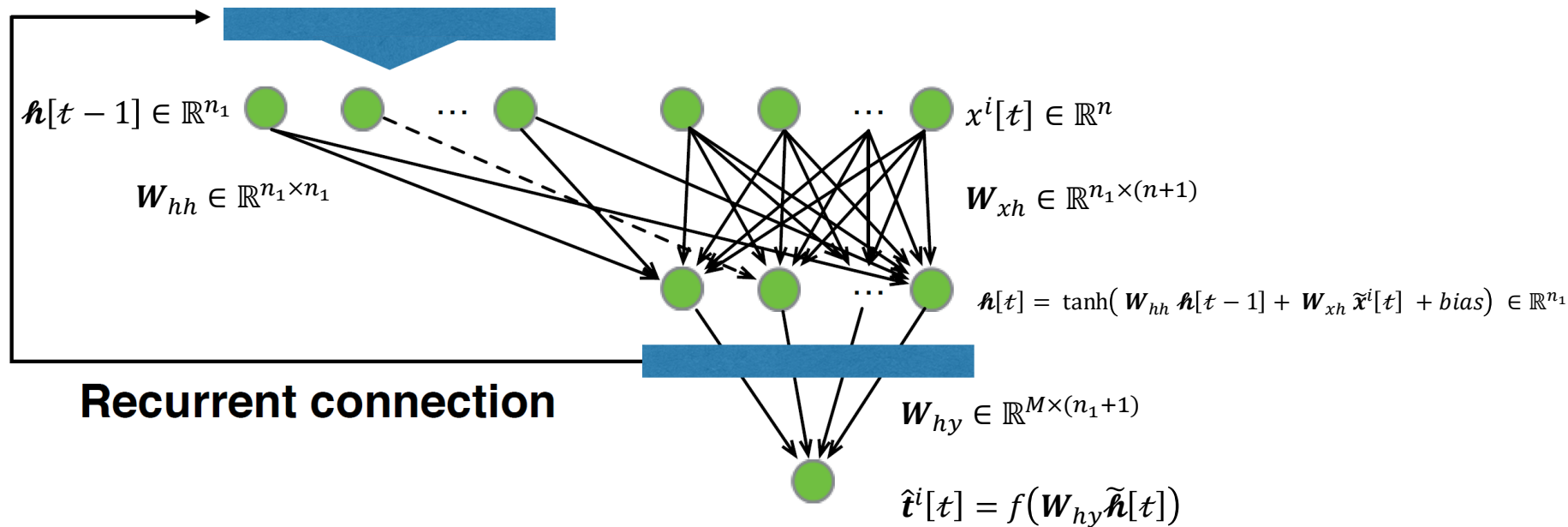
The same set of parameters ($\mathbf{W}_{hh}$, $\mathbf{W}_{xh}$, $\mathbf{W}_{hy}$) is used at each time step.

Simple Recurrent Networks (ii)



Credit: Andreas Maier – Deep Learning: Recurrent Neural Networks

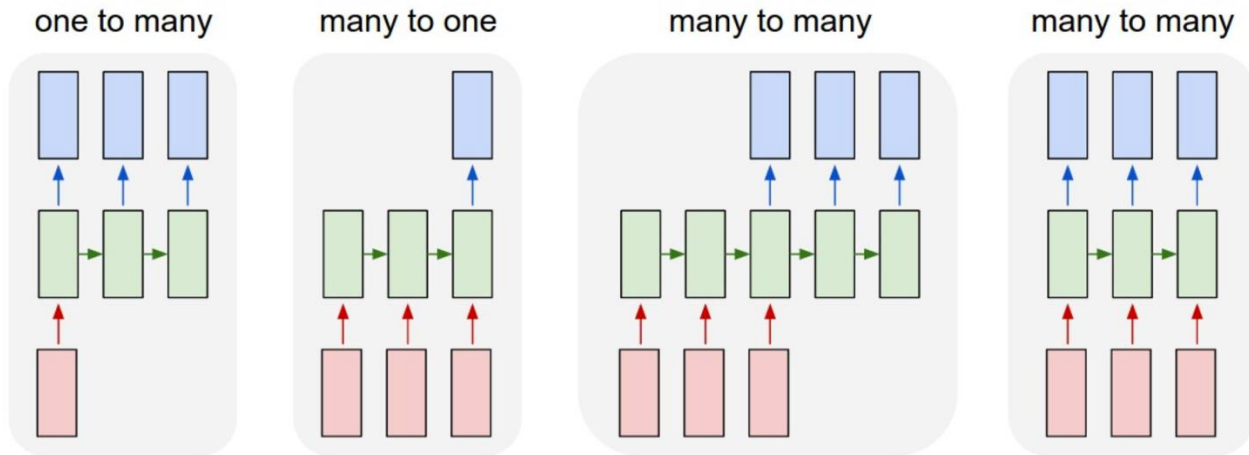
Simple Recurrent Networks (iii)



Hidden layer output is fed back into input layer.
Acts as a 'memory' layer (or memory 'cell').

Credit: Introduction to deep learning (ULB), Yann-Aël Le Borgne

RNN variations Input/outputs

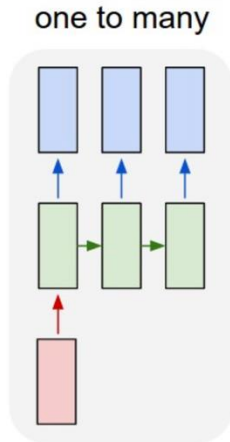


Credit: Andrej Karpathy

Example of applications: (i) Image captioning (ii) Sentiment analysis (iii) Machine translation (iv) Video frames classification

Any many other applications and architectures

Example of applications – Image captioning

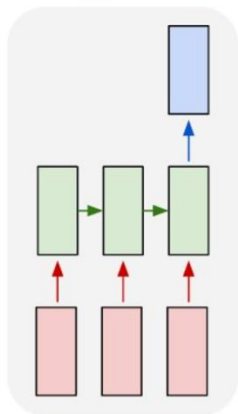


<start> a group of giraffes standing in a grassy area . <end>

Credit: <https://www.analyticsvidhya.com/blog/2018/04/solving-an-image-captioning-task-using-deep-learning/>

Example of applications – Sentiment analysis

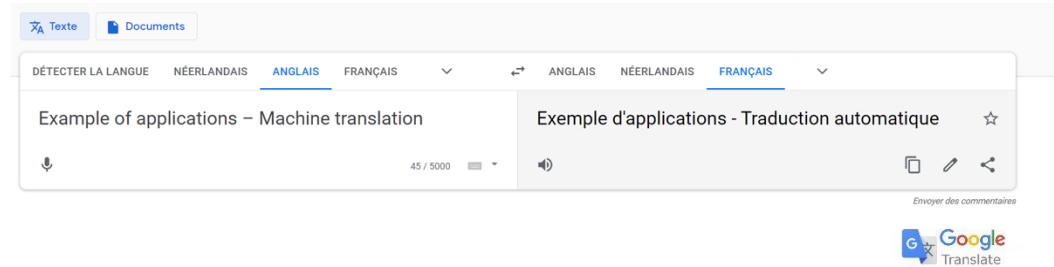
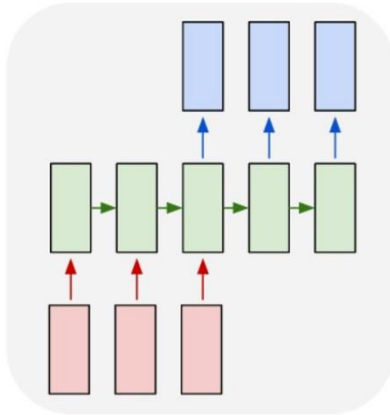
many to one



Loves the German bakeries in Sydney. Together with my imported honey it feels like home	Positive
@VivaLaLauren Mine is broken too! I miss my sidekick	Negative
Finished fixing my twitter...I had to unfollow and follow everyone again	Negative
@DinahLady I too, liked the movie! I want to buy the DVD when it comes out	Positive
@frugaldougal So sad to hear about @OscarTheCat	Negative
@Mofette brilliant! May the fourth be with you #starwarsday #starwars	Positive
Good morning thespians a bright and sunny day in UK, Spring at last	Positive
@DowneyisDOWNEY Me neither! My laptop's new, has dvd burning/ripping software but I just can't copy the files somehow!	Negative

Example of applications – Machine translation

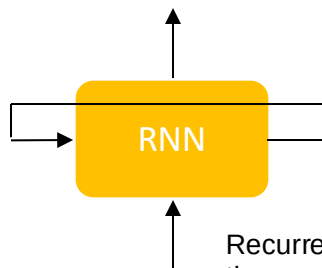
many to many



Recurrent Neural Networks properties

To model sequences, we need to:

- Handle variable-length sequences
 - E.g., no all sentences have the same length
- Model long-term dependencies
- Maintain information about order
- Share parameters across the sequence
 - By using weights sharing



Recurrent Neural Networks (RNN) meet these sequence modeling design criteria

To model sequences, we need to:

1. Handle variable-length sequences

E.g., not all sentences have the same length

Question: why is it not possible to use MLP?

Task: predicting the next word in a sentence

We had a three-course meal

We were hungry but cleaned the house before eating

Of all the places to travel, Mexico is at the top of my list

I love learning

2. Model long-term dependencies

We were hungry but cleaned the house before eating

Sometimes, the information needed to predict the next word occurs much earlier on.

To model sequences, we need to:

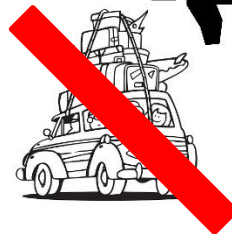
3. Maintain information about order



The trip was **good**, not **bad** at all

vs.

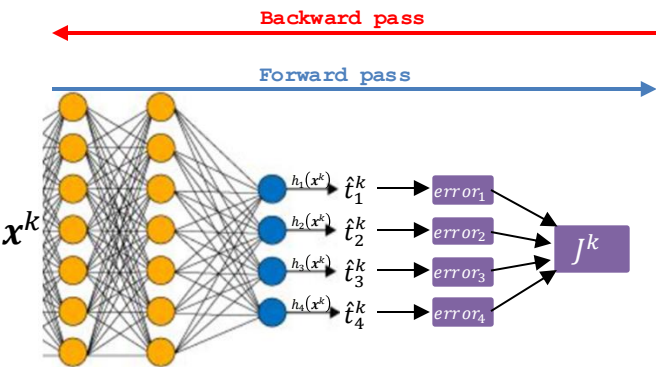
The trip was **bad**, not **good** at all



The same words but in a different order

Learning a Recurrent Neural Networks

Backpropagation through time (BPTT)



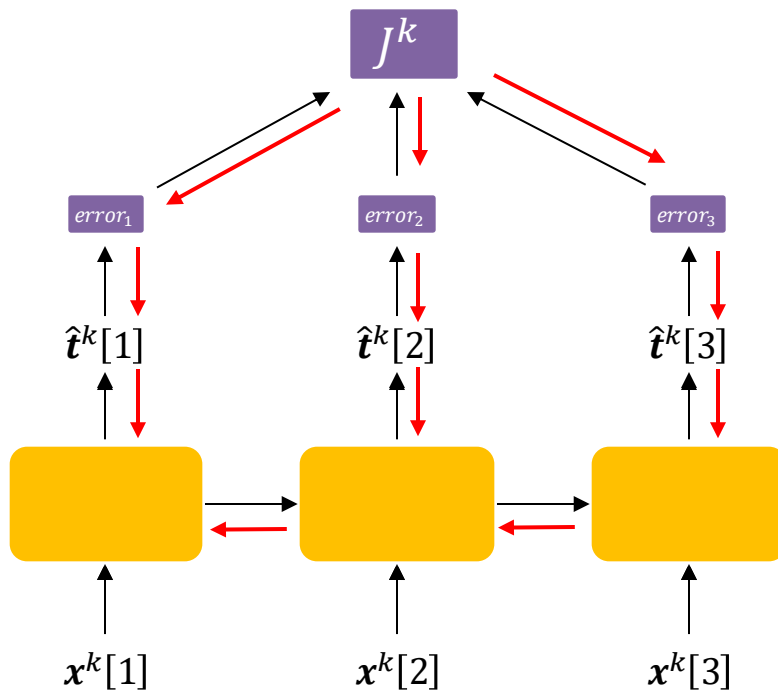
```

Initializes  $w(0)$ 
Repeat
  Select  $(x^k, t^k)$  from  $D_N$ 

  - Forward pass
    Compute output  $h(x^k)$ 

  - Backward pass
    Compute gradient  $\nabla_w J^k$ 

  - Update parameters
     $w(t+1) = w(t) - \alpha \nabla_w J^k$ 
    
```



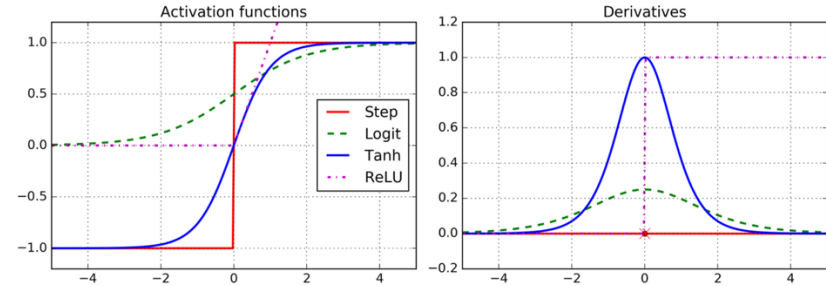
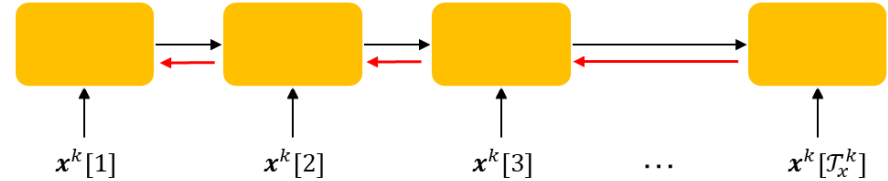
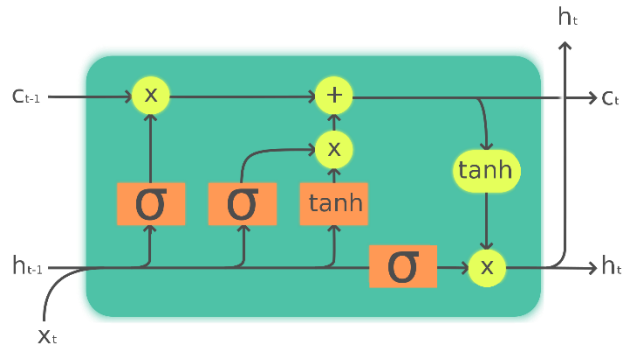
Learning a Recurrent Neural Networks

Backpropagation through time (BPTT)

Question for you: what could happen when $\mathcal{T}_x^k$ becomes big
→ Vanishing gradients
Bias parameters to capture short-term dependencies

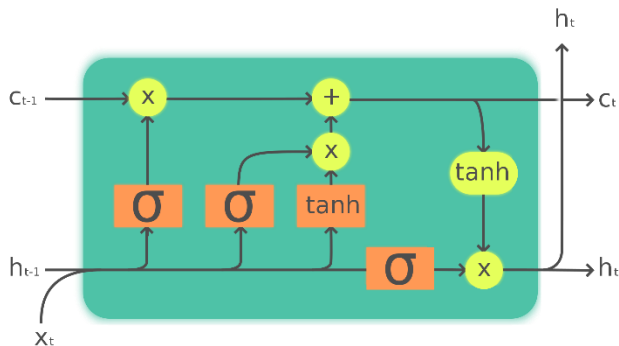
Solutions :

- Activation function (ReLU)
- New network architecture (Gated cells like LSTM)

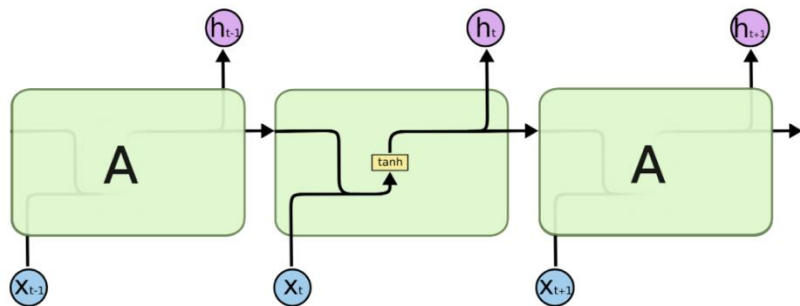


Gated cells: LSTM (Long short-term memory)

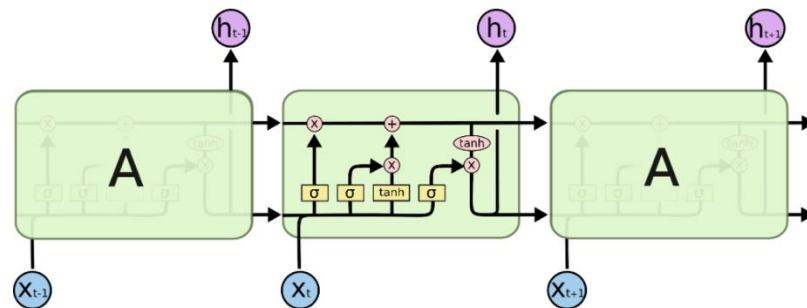
- LSTMs are explicitly designed to avoid the long-term dependency problem
- A more complex recurrent unit with gates to control what information is passed through
- Can more effectively track long-term dependencies in the data by controlling what information is passed through and what information is used to update the actual cell



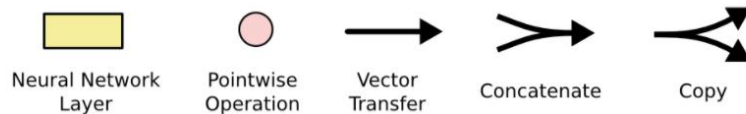
SRN <--> LSTM



Simple Recurrent Networks

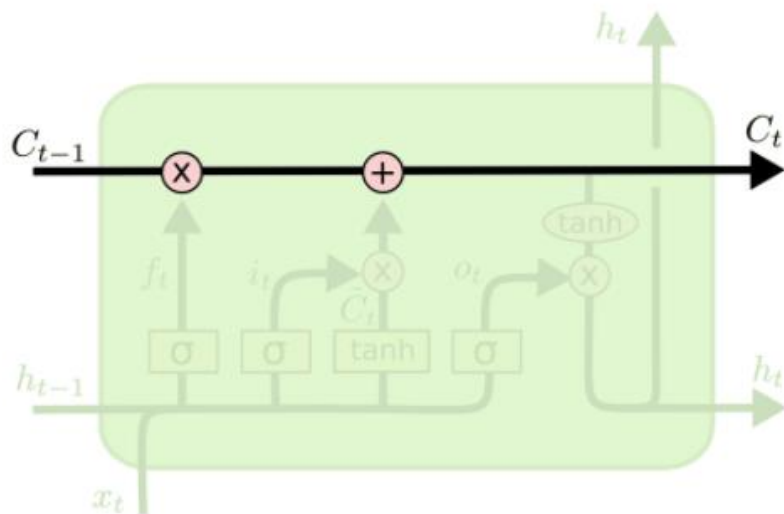


Long short-term memory



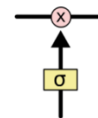
Credit: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTM - the cell state



The LSTM can remove or add information about the state of the cell, carefully regulated by structures called gates.

Gates are a way of letting information through in the cell state.

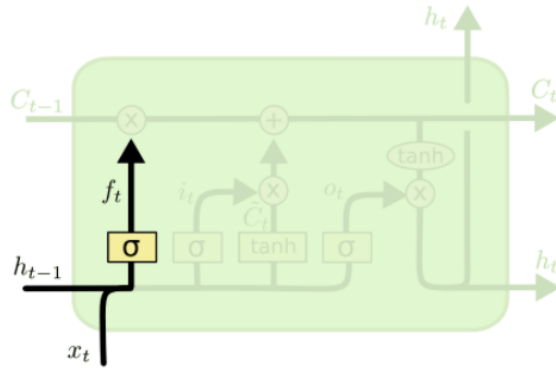


They are composed out of a sigmoid neural net layer and a pointwise multiplication operation.

An LSTM has three of these gates, to control the cell state.

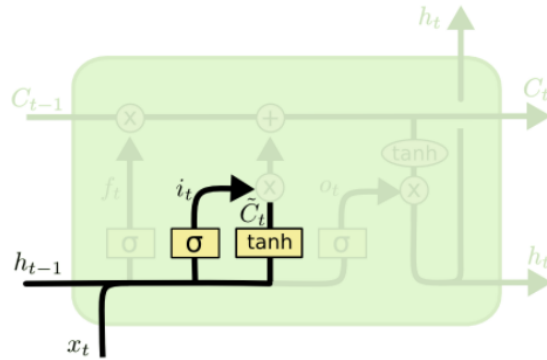
Question: why do you think we call it a “gate”?

LSTM – Step-by-step



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{"Forget gate layer."}$$

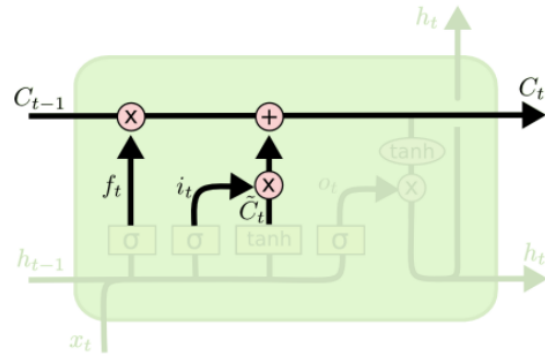
LSTM – Step-by-step



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{"Input gate layer"}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{New candidate values}$$

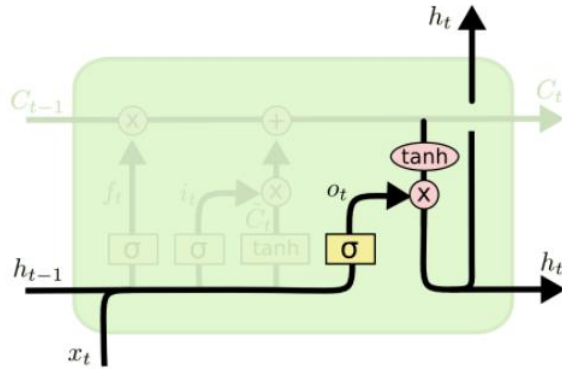
LSTM – Step-by-step



$$C_t = \overbrace{f_t * C_{t-1}}^{\text{Forget}} + \underbrace{i_t * \tilde{C}_t}_{\text{Add new information}}$$

Update the old cell state into the new cell state

LSTM – Step-by-step

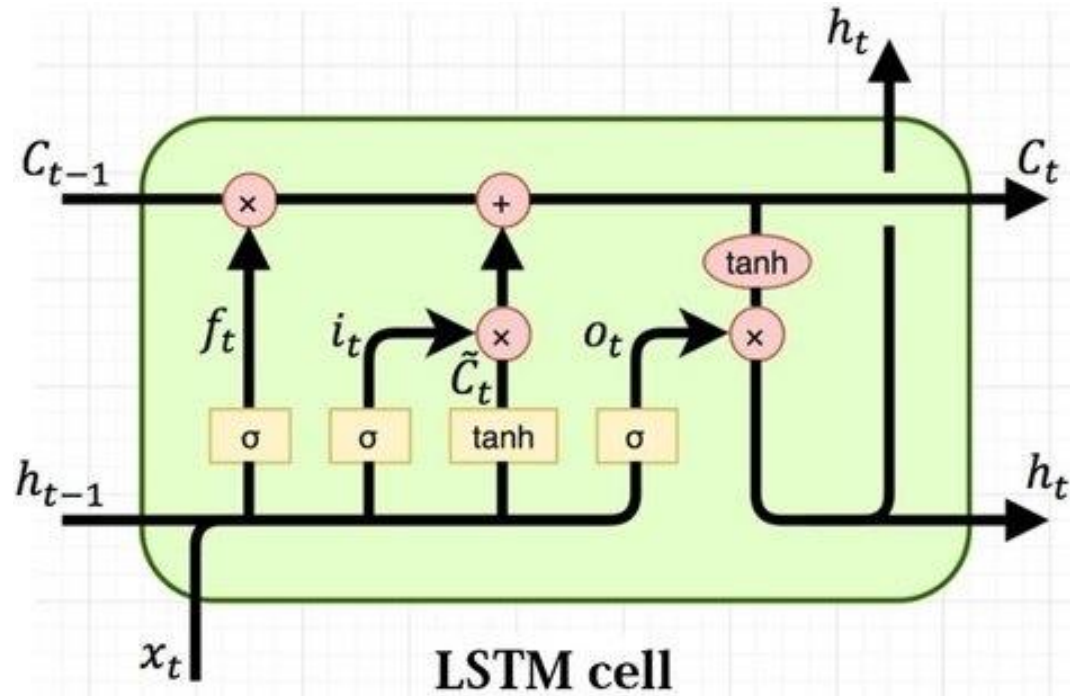


$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

We have to decide what to produce as output.

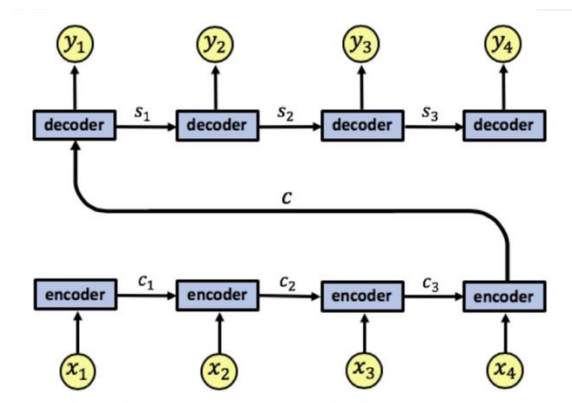
LSTM



Question for you: how did LSTM solve the vanishing problem?

Sequence-to-sequence (seq2seq)

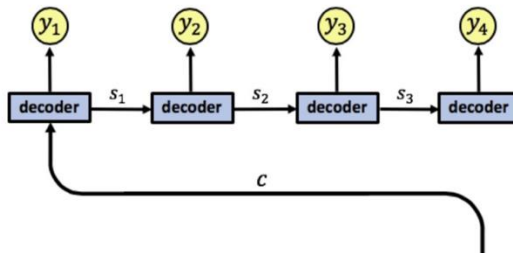
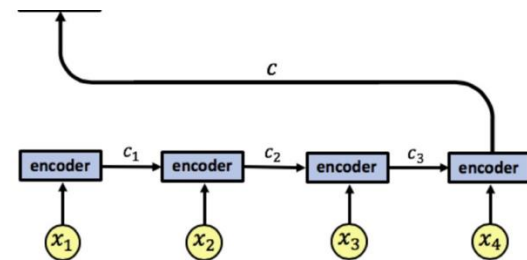
- SRNNs and LSTMs have been used widely for processing and making sense of sequential data.
- For seq2seq tasks, the usual approach is the encoder-decoder architecture.
- **Sequence-to-sequence (seq2seq)** models are used to convert type A sequences to type B sequences. For example, translating English sentences into French sentences is a sequence-to-sequence task.



The encoder has an input sequence $\mathbf{x1}, \mathbf{x2}, \mathbf{x3}, \mathbf{x4}$. We denote the encoder states by $\mathbf{c1}, \mathbf{c2}, \mathbf{c3}$. The encoder outputs a single output vector $\mathbf{c}$ which is passed as input to the decoder. Like the encoder, the decoder is also an RNN, we denote the decoder states by $\mathbf{s1}, \mathbf{s2}, \mathbf{s3}$ and the network's output by $\mathbf{y1}, \mathbf{y2}, \mathbf{y3}, \mathbf{y4}$.

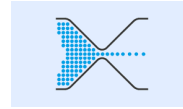
Sequence-to-sequence (seq2seq)

- In the encoding part, the states **c1**, **c2**, **c3** have access to information for whole input seen so far.
 - I.e.; the information flowing from the start in x_1 to the current stat x_t is available, in some modified/partial form, in the state c_t at time step t
- Therefore, the final state **c** of RNN (i.e.; *encoder*) must contain information for the entire input sequence.
- A major drawback of this architecture is that the encoding step must represent the entire input sequence **x1**, **x2**, **x3**, **x4** as a single vector **c**, which can lead to loss of information because all the information must be compressed in **c**.
- Moreover, the **decoder** must decipher the transmitted information from this single vector, a very complex task in itself.



Issues with RNN

- Encoding bottleneck



- Slow, no parallelization



- Not (really) long memory

